МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования «Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

«Основы баз данных»

Отчет о выполнении курсовой работы

«Учет заказов ателье индивидуального пошива»

Студент,
группы 5130201/40003 _____ Лебедевич К. О.

Руководитель,
преподаватель _____ Попов С.Г.

«23» 05 2026 г.

Санкт-Петербург, 2026

1/3. 02.04.26

2/3 05.05.26

Реферат

Предметной областью данной работы является «Учет заказов ателье индивидуального пошива». Основная проблема состоит в том, что с ростом числа клиентов у мастера усложняется ведение учёта заказов, а задача точно спрогнозировать срок выполнения и рассчитать расход материалов становится практически неразрешимой: без единой системы хранения данных о заказах, изделиях, используемых тканях и фурнитуре невозможно контролировать занятость портных и планировать закупки материалов.

В рамках работы спроектирована и реализована реляционная база данных, хранящая сведения об ателье, заказчиках, портных, администраторах, заказах, изделиях, а также об используемых тканях и фурнитуре. Сформированы и выполнены 9 запросов.

Для заполнения базы данных использовалась программа-генератор, написанная на языке C++ с применением библиотеки **libpq** для взаимодействия с PostgreSQL; СУБД PostgreSQL развёртывалась локально; диаграммы результатов запросов построены средствами Python (**matplotlib**).

Разработанная база данных может служить основой для построения систем учёта на предприятиях индивидуального пошива, платформ управления заказами, а также сервисов проката и ремонта одежды.

Содержание

Реферат **2**

Введение **4**

1 Постановка задачи **5**

2 Аналитика **6**

2.1 Описание предметной области 6
2.2 Цели функционирования базы данных 10
2.3 Перечень сущностей и атрибутов 10
2.4 Список ролей 11

3 Проектирование **12**

4 Лингвистическое описание схемы **16**

4.1 Конечная схема базы данных 16
4.2 Создание базы данных и заполнение значениями 19

5 Программирование **20**

5.1 Запрос №1.1 20
5.2 Запрос №1.2 22
5.3 Запрос №2 23
5.4 Запрос №3.1 25
5.5 Запрос №3.2 27
5.6 Запрос №4 29
5.7 Запрос №5 34
5.8 Запрос №6 37
5.9 Запрос №7 39
5.10 Запрос №8 41
5.11 Запрос №9 44

Заключение **45**

Список литературы **46**

6 Приложение А: Программа генерации базы данных **47**

7 Приложение Б: Программа генерации записей базы данных **50**

Введение

База данных — совокупность данных, хранимых как в соответствии со схемой данных, так и в произвольном порядке, манипулирование которыми выполняют в соответствии с правилами средств моделирования данных. Система управления базами данных — совокупность программных и лингвистических средств общего или специального назначения, обеспечивающих управление созданием и использованием баз данных.

Чтобы создать базу данных, сначала проектируют её структуру: определяют, какие данные хранить и как они связаны между собой. Затем схему переносят в СУБД и заполняют данными. Если база спроектирована правильно, данные не дублируются, запросы выполняются быстро и всё остаётся в порядке даже когда с базой работают одновременно несколько человек.

Данная работа посвящена проектированию и реализации реляционной базы данных для предметной области «Учет заказов ателье индивидуального пошива». Современное ателье обслуживает множество клиентов, и ведение учёта заказов, контроль расхода тканей и фурнитуры, а также хранение информации о пошиве каждого изделия и занятости портных требуют надёжной системы хранения и быстрого доступа к данным.

В качестве СУБД используется PostgreSQL; заполнение базы данных реализовано на языке C++ с использованием библиотеки **libpq**, а графическое представление результатов запросов построено средствами Python (**matplotlib**).

1 Постановка задачи

Цель работы: реализация функционирующей реляционной базы данных для предметной области «Учет заказов ателье индивидуального пошива» и выполнение заданных запросов.

Задачи работы:

- проанализировать предметную область;
- определить цели функционирования базы данных;
- выявить сущности, их атрибуты и связи;
- построить ER-диаграмму в нотации Чена;
- спроектировать схему базы данных и описать таблицы;
- перенести схему в СУБД PostgreSQL;
- реализовать программу заполнения базы данных тестовыми данными;
- выполнить запросы к базе данных;
- построить графическое представление результатов запросов.

2 Аналитика

2.1 Описание предметной области

Ателье индивидуального пошива — это предприятие, мастерская, где одежда изготавливается мастерами в соответствии с индивидуальными запросами каждого клиента, учитывая его особенности фигуры и снятые мерки. За счет этого достигается главное преимущество обращения в ателье — готовая вещь идеально сядет на человека любого телосложения. При пошиве изделий в ателье производство носит единичный и несерийный характер.
В рамках выполнения заказа индивидуального ателье можно выделить три роли: клиент (заказчик), исполнитель (портной) и администратор. Рассмотрим подробнее задачи и зоны ответственности каждой из ролей.

Клиент или заказчик — физическое лицо, инициирующее сам процесс пошива изделия. Заказчик также является конечным потребителем изготовленного изделия и источником требований к его выполнению. На этапе обращения клиент ответственен за формирование задачи и требований к конечному продукту.

Администратор — сотрудник предприятия, обеспечивающий организационное сопровождение процесса пошива. Администратор выполняет функцию приема и регистрации клиента, фиксирует его контактные данные, информирует об услугах и ценах, после чего передает клиента портному для дальнейшей консультации. Зачастую в настоящее время встречается практика, при которой один сотрудник ателье выполняет функции как портного, так и администратора одновременно.

Портной или исполнитель — сотрудник, непосредственно ответственный за выполнение работы по пошиву изделия. Портной несет ответственность за качество посадки изделия по фигуре, соблюдение сроков выполнения заказа и соответствие изготовленного изделия пожеланиям клиента.

Рассмотрим процесс формирования и выполнения заказа, являющийся основным производственным циклом. Процесс начинается с обращения клиента в ателье при возникновении у того потребности в услуге индивидуального пошива. Администратор фиксирует контактные данные клиента, такие как ФИО и номер телефона, информирует клиента о перечне предоставляемых услуг и соответствующих расценках, назначает свободного портного для выполнения пошива и консультации и сопровождает к нему заказчика. Далее происходит консультация с портным. На этом этапе портной помогает заказчику определиться с желаемым результатом, в ходе чего заказчик и портной совместно формируют характеристики будущего изделия: его тип (пиджак, брюки, юбка), цвет, материал и количество тканей и фурнитуры, особенности кроя (прямой, зауженный, свободный). Для изготовления изделий могут быть использованы ткани всех возможных составов (шелк, лен, шерсть, кашемир, мохер, хлопок, ткани с добавлением синтетических материалов), включая более редкие и экзотические варианты, например может быть использована альпака или крапива. Ткань обычно не изготавливается самим

ателье, а закупается у поставщиков, зачастую итальянских или английских текстильных домов, таких как Vitale Barberis Canonico, Holland&Sherry, Loro Piana и многих других. Также текстильный дом или фабрика предоставляет ателье специальную книжку с образцами своих тканей, просматривая которую, заказчик может тактильно ознакомиться с каждым вариантом и выбрать подходящий для себя. Для определения размера изделия и требуемого метража отрезов ткани также снимаются параметры фигуры заказчика, в полном объеме это: обхваты шеи, груди, талии и бедер; ширина плеч, длины конечностей. Эти параметры при занесении их в каталог могут также быть использованы при дальнейших заказах, если измерения клиента остались неизменными. Если ткань для изготовления изделия не предоставляется клиентом и приобретается в ателье, фиксируется списание со склада с указанием артикула ткани, цвета и метража. Аналогичное списание необходимо оформить и с фурнитурой, не предоставляемой заказчиком. После согласования всех деталей заказа с клиентом администратор формирует заказ, где фиксируется: номер работы, дата приема, перечень предоставляемых услуг и соответствующих расценок, данные клиента и портного, а также ориентировочный срок первой примерки изделия. Зафиксированные данные заносятся в квитанцию, одну копию которой получает клиент, а другую копию — предприятие, после чего заказ принимается в работу. При индивидуальном пошиве портные практически не используют готовые лекала, однако иногда, например, в целях экономии может быть взята готовая выкройка (бумажный или электронный документ, представляющий чертеж с лекалами и подписанными затратами ткани), которая корректируется под индивидуальные мерки клиента. В другом случае, портной разрабатывает лекала с нуля, прежде всего выполняя чертеж на специальной жесткой бумаге для изготовления лекал, после чего полученная вырезка может храниться в ателье и быть использована для дальнейшего изготовления изделий для клиента. Портной далее выполняет расклад лекал на ткани, обводит их специальным мелком, стремясь минимизировать обрезки и вырезает, используя портновские ножницы. Портному на этом этапе необходимо вести учет фактических затрат ткани, сравнивая который с плановым, указать о необходимости дополнительного списания или, наоборот, возврата на склад. Оставшиеся после раскроя обрезки могут быть вручены заказчику для облегчения ремонта изделия. Далее выполняется пошив изделия: стачивание деталей машинной или ручной строчкой, обработка швов, пришивание карманов, бортовки или подклада при необходимости. На этом этапе необходимо держать в уме, что каждому шву в будущем, по желанию клиента, может потребоваться корректировка, поэтому используется специальная толстая белая нить, называемая наметочной. Также фиксируется фактическое время портного при работе над заказом. Когда черновой вариант готов, администратор связывается с клиентом для согласования даты первой примерки, согласованная дата также фиксируется. На данном этапе клиент совместно с портным выявляют дефекты пошива изделия или несовершенства посадки по фигуре заказчика, составляется перечень необходимых

доработок и при необходимости назначается вторая примерка. Если клиент удовлетворен изделием, наступает этап финальной обработки — паровая обработка, чистка от ниток. Администратор уведомляет клиента о готовности заказа и при отсутствии дальнейших замечаний заказчика происходит расчет и заказ считается завершенным. Клиент получает изделие и сохраняет квитанцию о заказе, которая может быть использована при обращении в ателье в последующие 14 недель для неоплачиваемого внесения корректировок, если в таковых есть потребность по вине исполнителя. В случае, если клиент остался не удовлетворен изготовленным изделием, он может отказаться вносить оплату и изделие останется в собственности ателье.

Далее на рисунке 1 представлена составленная в соответствии с описанием предметной области ER-диаграмма, отражающая следующие отношения:

- Администратор работает в ателье.
- Портной работает в ателье.
- Портной подсчитывает затраченное время.
- Портной осуществляет пошив изделия
- Заказчик обращается в ателье.
- Администратор фикасирует данные заказчика.
- Заказчик и портной определяют параметры изделия.
- Изделие является частью заказа.
- Ткань является частью изделия.
- Фурнитра является частью изделияю
- Администратор организационно сопровождает заказ.

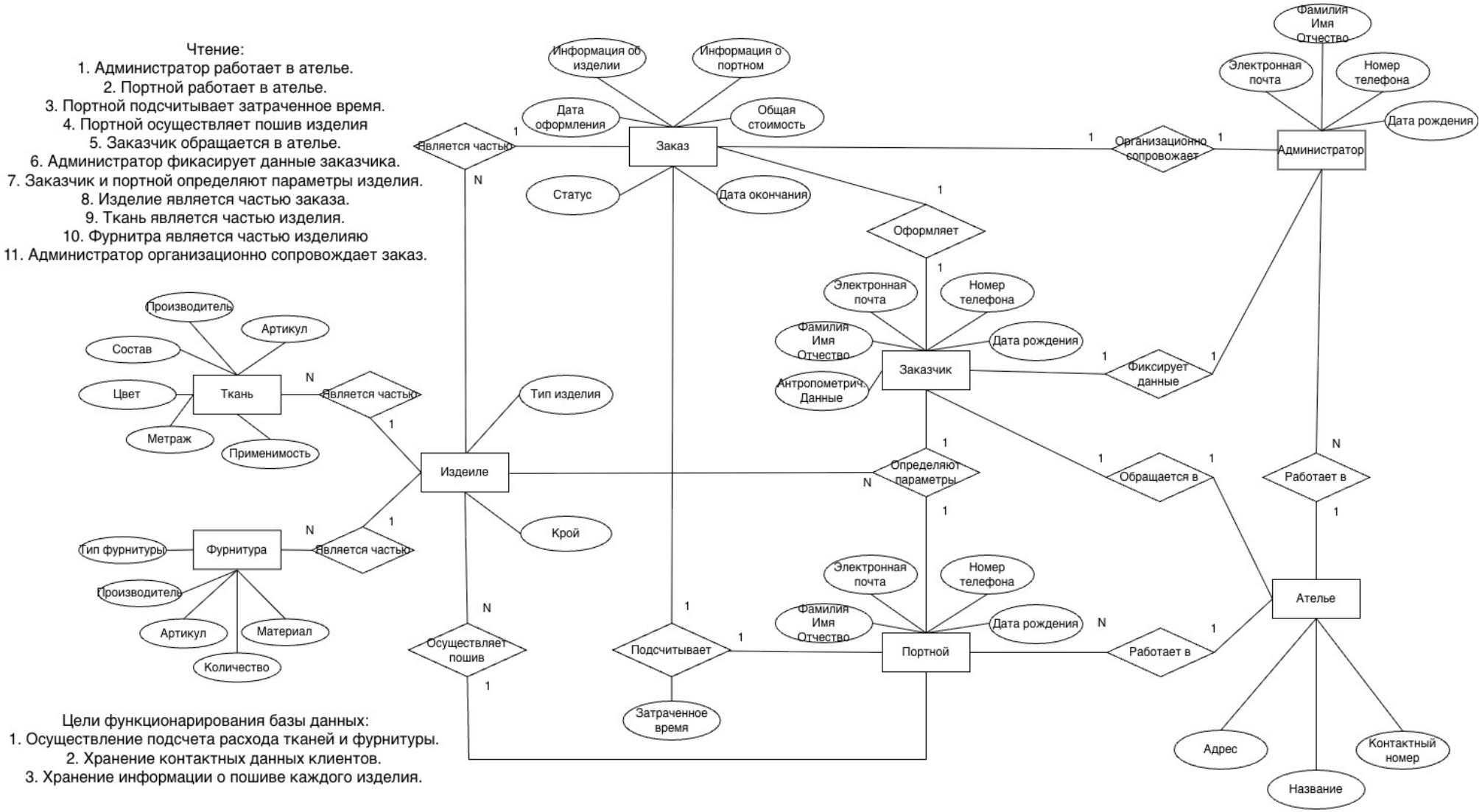


Рис. 1: ER диаграмма базы данных.

Далее на рисунке 2 представлена схема связи объектов.

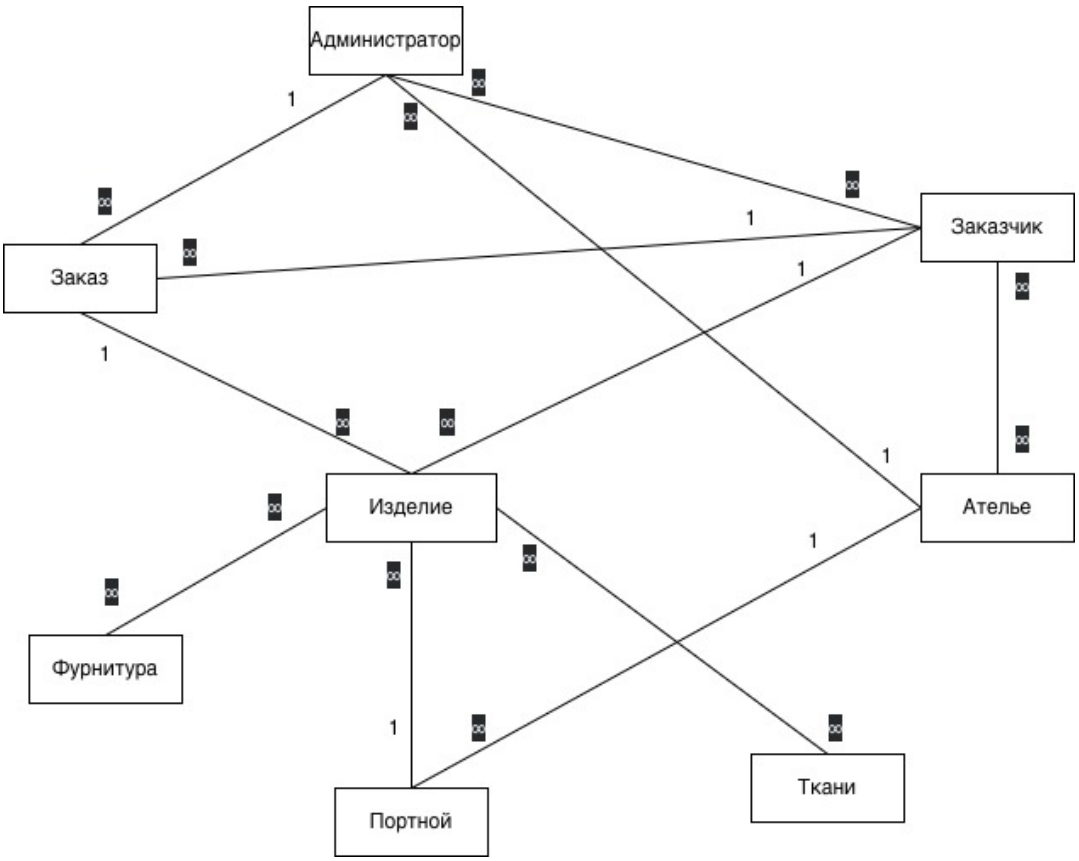


Рис. 2: Схема связи объектов.

2.2 Цели функционирования базы данных

1. Осуществление подсчета расхода тканей и фурнитуры.
2. Хранение контактных данных клиентов.
3. Хранение информации о пошиве каждого изделия.

2.3 Перечень сущностей и атрибутов

1. **Ателье** — предприятие, оказывающее услуги по пошиву и ремонту одежды на заказ. Атрибуты: название ателье, адрес, контактный телефон.
2. **Заказчик** — лицо, обратившееся в ателье для изготовления или ремонта изделия. Атрибуты: фамилия имя отчество, номер телефона, электронная почта, дата рождения, антропометрические данные.
3. **Портной** — сотрудник ателье, непосредственно выполняющий работы по раскрою и пошиву изделий. Атрибуты: фамилия имя отчество, номер телефона, электронная почта, дата рождения.

4. **Администратор** — сотрудник, осуществляющий прием и регистрацию заказов, запись на примерки и расчеты с клиентами. Атрибуты: фамилия имя отчество, номер телефона, электронная почта, дата рождения.

5. **Заказ** — совокупность данных о конкретном обращении клиента, включающая перечень работ и сроки выполнения. Атрибуты: номер заказа, дата оформления, дата окончания или прочерк, статус заказа, общая стоимость, информация об изделии.

6. **Изделие** — результат работы портного, описание модели одежды с указанием особенностей кроя и материалов. Атрибуты: тип изделия (платье, брюки, пиджак и т.д.), крой (прямой, зауженный, свободный), перечень характеристик ткани и площади отрезов, перечень характеристик фурнитуры и ее количества.

7. **Ткань** — материал, используемый для пошива изделия, с описанием его свойств и происхождения. Атрибуты: применимость, метраж, артикул, производитель, состав, цвет.

8. **Фурнитура** — вспомогательные элементы (пуговицы, молнии, нитки), необходимые для изготовления и отделки изделия. Атрибуты: материал, количество, артикул, тип фурнитуры, производитель.

2.4 Список ролей

1. **Заказчик** — физическое лицо, обратившееся в ателье с целью изготовления нового изделия, ремонта или переделки существующего. Заказчик формулирует требования к будущему изделию, предоставляет исходные данные, участвует в примерках и принимает готовую работу.

2. **Портной** — сотрудник ателье, непосредственно выполняющий технологические операции по раскрою, пошиву и отделке изделий. Портной снимает мерки, строит лекала, проводит примерки, вносит корректировки и отвечает за качество посадки готового изделия на фигуре.

3. **Администратор** — сотрудник, обеспечивающий организационное сопровождение производственного цикла. Администратор регистрирует клиентов, оформляет заказы, ведет запись на примерки, контролирует сроки выполнения, принимает оплату и выдает готовые заказы.

3 Проектирование

- **VARCHAR(40)** для фамилии — выбрано с учетом возможности существования двойных и составных русских фамилий (например, Салтыков-Щедрин, Мамин-Сибиряк), максимальная длина которых в редких случаях достигает 35-38 символов. Запас в 2-5 символов обеспечивает корректное хранение даже самых длинных фамилий.

- **VARCHAR(15)** для имени — установлено на основе анализа самых длинных русских имен (например, Аполлинарий, Александра, Вениамин, Максимилиан), которые содержат не более 12-13 символов. Дополнительный запас в 2-3 символа учитывает возможные редко встречающиеся варианты.

- **VARCHAR(20)** для отчества — определено с учетом того, что отчества образуются от имен путем добавления суффиксов (-вич, -вна, -чна, -ична), что увеличивает длину на 3-5 символов относительно исходного имени. Например, Максимилианович содержит 17 символов.

- **VARCHAR(15)** для телефонного номера — выбрано с учетом стандартного российского формата +7XXXXXXXXXX (12 символов). Возможность хранения дополнительных символов (скобки, дефисы) требует расширения до 15 символов.

- **VARCHAR(320)** для электронной почты — установлено на основе максимальной допустимой длины email-адреса (64 символа для локальной части и 255 для домена).

- **VARCHAR(200)** Для хранения адреса предприятия ателье. Значение максимальной допустимой длины основано на реальных адресах с учетом запаса символов. Пример адреса: "Конногвардейский б-р, 4, Санкт-Петербург, 190000".

- **VARCHAR(75)** Для хранения адреса состава ткани. Значение максимальной допустимой длины основано на примере состава из 5 материалов с учетом запаса символов. Пример состава: "Кашемир 25 Лиоцелл 25 полиэстер 20 полиамид 3 эластан 2".

- **VARCHAR(20)** для артикулов и типов фурнитуры — выбрано на основе анализа реальных артикулов производителей тканей. Пример артикула текстильного дома Vitale Barberis Canonico: 10.712/333.

- **VARCHAR(25)** для цвета — установлено с учетом возможности указания сложных оттенков (например, «серо-голубой», «темно-бордовый»).

Таблица 1: Ателье - Atelier

Атрибут (рус)	**Атрибут (eng)**	**Тип**	**По умолчанию**	**Ключ**	**Ссылка**
Ателье_id	Atelier_id	INT	NOT NULL	PK	
Адрес	Address	VARCHAR(200)	NOT NULL		
Телефон	Phone	VARCHAR(15)	NOT NULL		

Таблица 2: Заказчик - Client

Атрибут (рус)	**Атрибут (eng)**	**Тип**	**По умолчанию**	**Ключ**	**Ссылка**
Заказчик_id	Client_id	INT	NOT NULL	PK	
Фамилия	Surname	VARCHAR(40)	NOT NULL		
Имя	Name	VARCHAR(15)	NOT NULL		
Отчество	Patronymic	VARCHAR(20)	NULL		
Телефон	Phone	VARCHAR(15)	NOT NULL		
Эл_почта	Email	VARCHAR(320)	NULL		
Дата_рождения	Birth_date	DATE	NOT NULL		

Таблица 3: Ателье_Заказчик - Atelier_Client

Атрибут (рус)	**Атрибут (eng)**	**Тип**	**По умолчанию**	**Ключ**	**Ссылка**
Заказчик_id	Client_id	INT	NOT NULL	FK	Client
Ателье_id	Atelier_id	INT	NOT NULL	FK	Atelier

Таблица 4: Администратор - Administrator

Атрибут (рус)	**Атрибут (eng)**	**Тип**	**По умолчанию**	**Ключ**	**Ссылка**
Админ-р_id	Admin_id	INT	NOT NULL	PK	
Фамилия	Surname	VARCHAR(40)	NOT NULL		
Имя	Name	VARCHAR(15)	NOT NULL		
Отчество	Patronymic	VARCHAR(20)	NULL		
Телефон	Phone	VARCHAR(15)	NOT NULL		
Эл_почта	Email	VARCHAR(320)	NOT NULL		
Дата_рождения	Birth_date	DATE	NOT NULL		
Ателье_id	Atelier_id	INT	NOT NULL	FK	Atelier

Таблица 5: Портной - Tailor

Атрибут (рус)	**Атрибут (eng)**	**Тип**	**По умолчанию**	**Ключ**	**Ссылка**
Портной_id	Tailor_id	INT	NOT NULL	PK	
Фамилия	Surname	VARCHAR(40)	NOT NULL		
Имя	Name	VARCHAR(15)	NOT NULL		
Отчество	Patronymic	VARCHAR(20)	NULL		
Телефон	Phone	VARCHAR(15)	NOT NULL		
Эл_почта	Email	VARCHAR(320)	NULL		
Дата_рождения	Birth_date	DATE	NOT NULL		
Ателье_id	Atelier_id	INT	NOT NULL	FK	Atelier

Таблица 6: Заказ - Order

Атрибут (рус)	**Атрибут (eng)**	**Тип**	**По умолчанию**	**Ключ**	**Ссылка**
Заказ_id	Order_id	INT	NOT NULL	PK	
Дата_оформ-я	Order_date	DATE	NOT NULL		
Дата_окон-я	Completion_date	DATE	NULL		
Статус	Status	ENUM	'Принят'		
Общ._стоимость	Total_cost	DECIMAL(10,2)	0.00		
Заказчик_id	Client_id	INT	NOT NULL	FK	Client
Портной_id	Tailor_id	INT	NOT NULL	FK	Tailor
Админ-р_id	Admin_id	INT	NOT NULL	FK	Admin

Таблица 7: Изделие - Garment

Атрибут (рус)	**Атрибут (eng)**	**Тип**	**По умолчанию**	**Ключ**	**Ссылка**
Изделие_id	Garment_id	INT	NOT NULL	PK	
Тип_изделия	Product_type	ENUM	NOT NULL		
Крой	Cut_style	ENUM	NOT NULL		
Заказ_id	Order_id	INT	NOT NULL	FK	Order
Портной_id	Tailor_id	INT	NOT NULL	FK	Tailor

Таблица 8: Ткань - Fabric

Атрибут (рус)	**Атрибут (eng)**	**Тип**	**По умолчанию**	**Ключ**	**Ссылка**
Ткань_id	Fabric_id	INT	NOT NULL	PK	
Артикул	Article	VARCHAR(20)	NOT NULL		
Производитель	Manufacturer	VARCHAR(60)	NOT NULL		
Состав	Composition	VARCHAR(75)	NOT NULL		
Метраж	Meters_used	DECIMAL(10,2)	NOT NULL		
Цвет	Color	VARCHAR(25)	NOT NULL		
Применимость	Applicability	ENUM	'Основная'		

Таблица 9: Ткань_Изделие - Fabric_Garment

Атрибут (рус)	**Атрибут (eng)**	**Тип**	**По умолчанию**	**Ключ**	**Ссылка**
Изделие_id	Garment_id	INT	NOT NULL	FK	Garment
Ткань_id	Fabric_id	INT	NOT NULL	FK	Fabric

Таблица 10: Фурнитура - Accessories

Атрибут (рус)	**Атрибут (eng)**	**Тип**	**По умолчанию**	**Ключ**	**Ссылка**
Фурнитура_id	Accessory_id	INT	NOT NULL	PK	
Тип	Type	VARCHAR(30)	NOT NULL		
Материал	Material	VARCHAR(40)	NOT NULL		
Производитель	Manufacturer	VARCHAR(60)	NOT NULL		
Артикул	Article	VARCHAR(20)	NOT NULL		
Количество	Quantity_used	INT	NOT NULL		

Таблица 11: Фурнитура_Изделие - Accessories_Garment

Атрибут (рус)	**Атрибут (eng)**	**Тип**	**По умолчанию**	**Ключ**	**Ссылка**
Изделие_id	Garment_id	INT	NOT NULL	FK	Garment
Фурнитура_id	Accessory_id	INT	NOT NULL	FK	Accessories

4 Лингвистическое описание схемы

1. Ателье (Ателье_id(int), Адрес(varchar(200)), Контактный_номер(varchar(15))).
2. Заказчик (Заказчик_id(int), Фамилия(varchar(40)), Имя(varchar(15)), Отчество(varchar(20)), Телефон(varchar(15)), Электронная_почта(varchar(320)), Дата_рождения(date)).
3. Ателье_Заказчик (Заказчик_id(int), Ателье_id(int)).
4. Администратор (Администратор_id(int), Фамилия(varchar(40)), Имя(varchar(15)), Отчество(varchar(20)), Телефон(varchar(15)), Электронная_почта(varchar(320)), Дата_рождения(date), Ателье_id(int)).
5. Портной (Портной_id(int), Фамилия(varchar(40)), Имя(varchar(15)), Отчество(varchar(20)), Телефон(varchar(15)), Электронная_почта(varchar(320)), Дата_рождения(date), Ателье_id(int)).
6. Заказ (Заказ_id(int), Дата_оформления(date), Дата_окончания(date), Статус(enum), Общая_стоимость(decimal(10,2)), Заказчик_id(int), Портной_id(int)).
7. Изделие (Изделие_id(int), Тип_изделия(enum), Крой(enum), Заказ_id(int), Портной_id(int)).
8. Ткань (Ткань_id(int), Артикул(varchar(20)), Производитель(varchar(60)), Состав(varchar(75)), Метраж(decimal(10,2)), Цвет(varchar(25)), Применимость(enum)).
9. Ткань_Изделие (Изделие_id(int), Ткань_id(int)).
10. Фурнитура (Фурнитура_id(int), Тип(varchar(30)), Материал(varchar(40)), Производитель(varchar(60)), Артикул(varchar(20)), Количество(int)).
11. Фурнитура_Изделие (Изделие_id(int), Фурнитура_id(int)).

4.1 Конечная схема базы данных

Далее на Рис. 3 и Рис. 4 представлены конечные схемы базы данных на русском и английском языке соответственно.

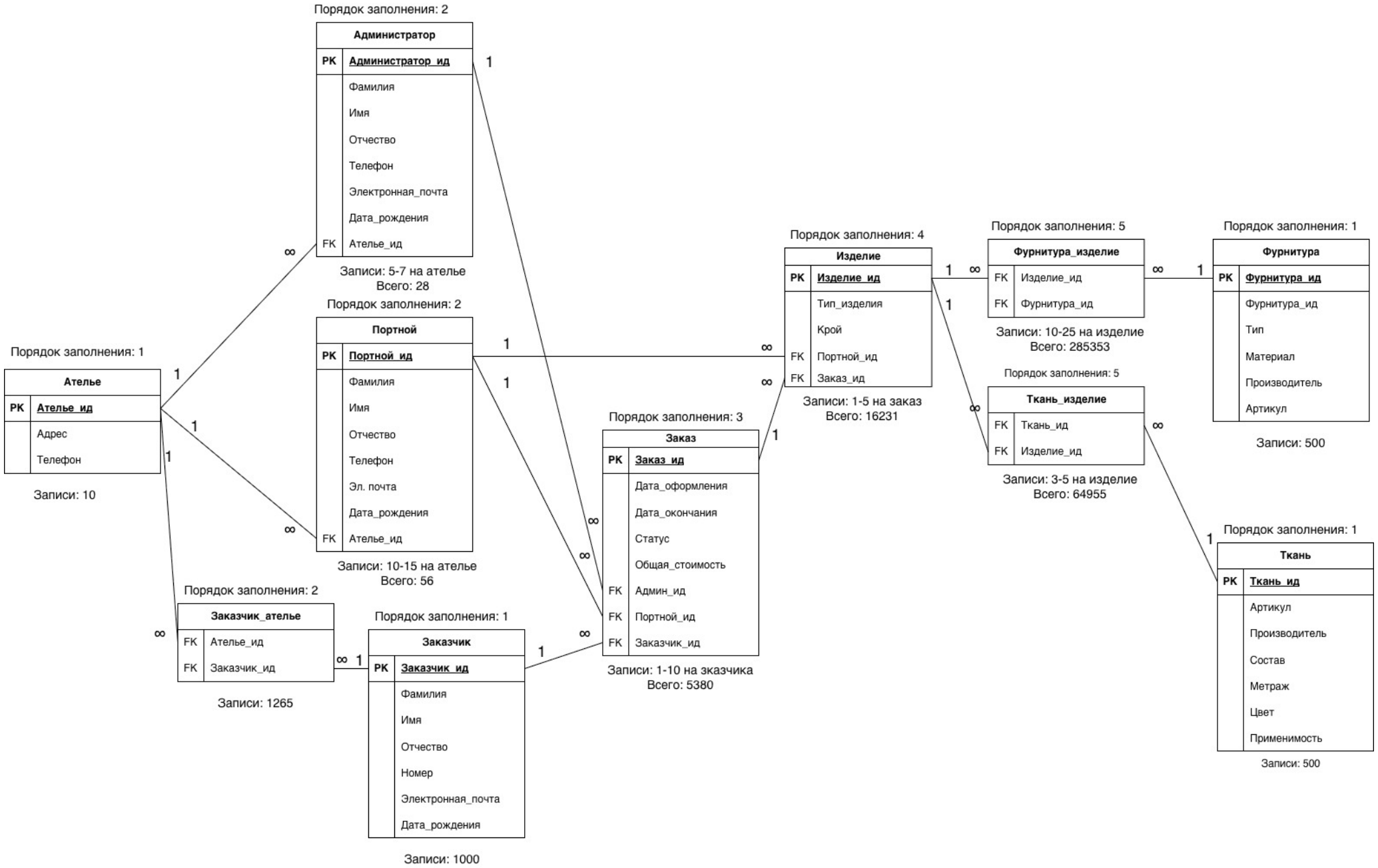


Рис. 3: Схема базы данных на русском.

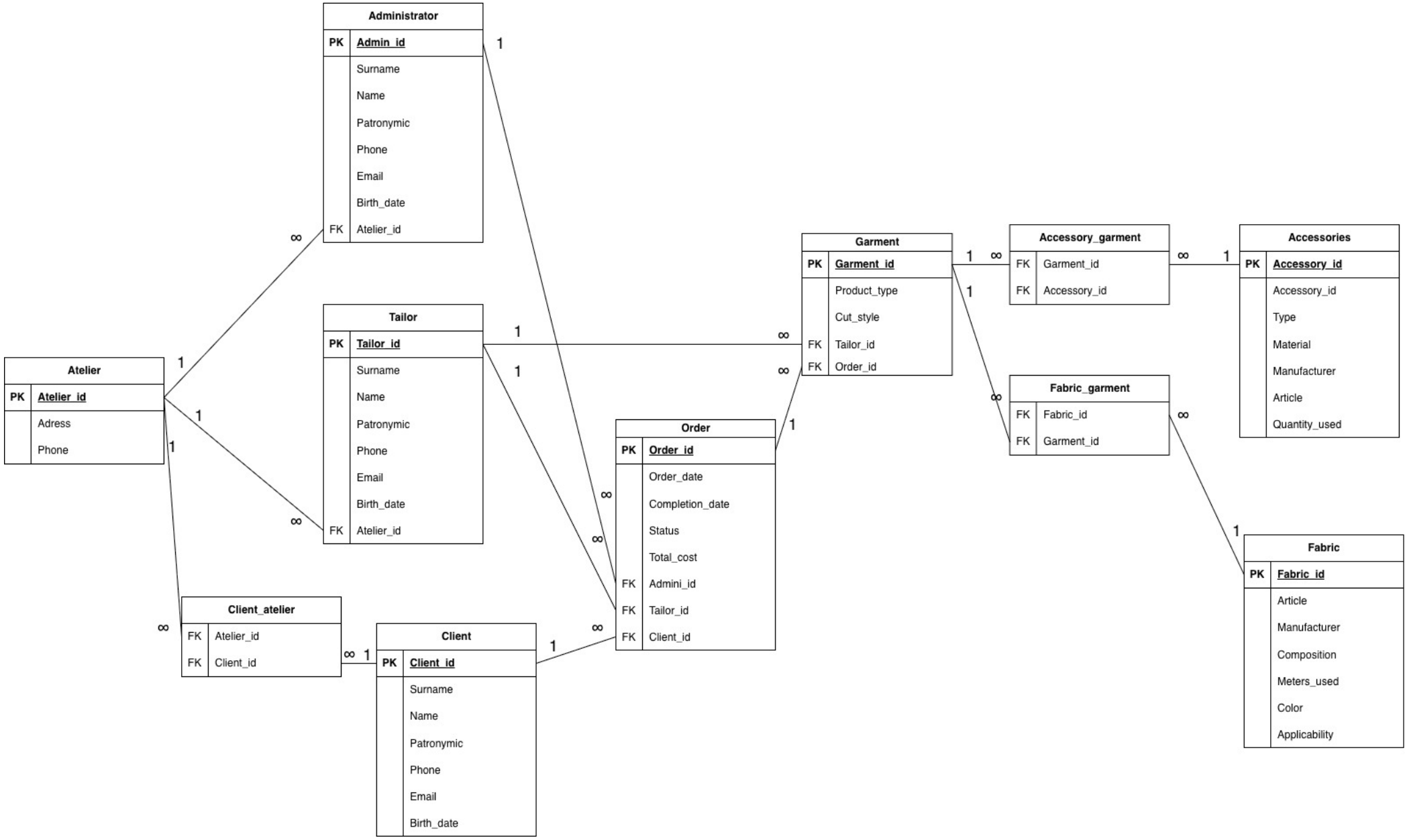


Рис. 4: Схема базы данных на английском.

4.2 Создание базы данных и заполнение значениями

База данных и таблицы, содержащиеся в ней, были созданы с помощью SQL-запросов и командной строки PostgreSQL. Для обеспечения целостности данных были настроены внешние ключи с каскадным удалением, а также созданы составные первичные ключи для таблиц-связок. Код создания базы данных представлен в приложении А.

База данных была заполнена случайными значениями, сгенерированными программой на языке C++. Для подключения к базе данных использовалась библиотека `libpq` — официальный клиентский интерфейс PostgreSQL. Заполнение выполнялось последовательно: сначала заполнялись независимые таблицы-словари, затем таблицы, содержащие внешние ключи. Всего было выполнено 9 этапов заполнения для 11 таблиц. В результате база данных содержит следующее количество записей, представленное в таблице 12.

Таблица 12: Количество записей в таблицах базы данных

Таблица	**Количество записей**
Ателье (atelier)	10
Портные (tailor)	113
Администраторы (administrator)	58
Заказчики (client)	1 000
Ателье_Заказчик (atelier_client)	1 282
Заказы (order)	5 468
Изделия (garment)	32 228
Ткани (fabric)	500
Фурнитура (accessories)	500
Ткань_Изделие (fabric_garment)	193 769
Фурнитура_Изделие (accessories_garment)	848 373
Итого	**1 083 301**

Код заполнения таблиц базы данных представлен в приложении Б.

Лист проверки выполнения запросов курсовой работы по дисциплине «Теоретические основы баз данных» 4 семестр

Имя, отчество, фамилия Кирюшин Олег Викторович Группа 5130201/40003

Тема работы Учет заказов ателье индивидуального пошива

№ п.п.	текст запроса	принято	
		запрос	отчет
1.1	Найти заказ-ов, кот. пос. в ателье А, вып для заказч. Б, в кот исп. ткань С и ткань D	+	+
2	Для ателье А пос число заказов, поступ. пошив. Б	+	+
3	Найти заказчиков, кот заказали во всех ателье макс. количество изд. фурнитуры	+	+
4	Посчитать число заказчиков с одинаковым числом изделий. пост 2ф листа	+	+
5	Для каждого посчитать число заказов и число изделий (использовать 2ф)	+	+
6	Найти тип фурнитуры, кот. использовались чаще, чем фурнитура А	+	+
7	Найти заказчиков, котор. никогда не использовали ткань А	+	+
8	Для каждого ателье и каждого администратора, пост. 3ф лист	+	+
9	Для изделия А поменять фурнитуру А на фурнитуру Б	+	+
10			

1.2 1.1... Не менее двух пар +

3.2 минимальное +

5 Программирование

В данном разделе представлены SQL-запросы к спроектированной базе данных. Для каждого запроса приводится текстовое описание решаемой задачи, описание алгоритма выполнения, реляционная формула, листинг на языке SQL, результат выполнения.

5.1 Запрос №1.1

Необходимо найти администраторов, работающих в ателье А, выполнявших заказ для заказчика Б, в котором используется фурнитура В и ткань Г.

Текстовое объяснение: при помощи `INNER JOIN` соединяются таблицы `administrator`, `"order"`, `garment`, `fabric_garment`, `fabric`, `accessories_garment` и `accessories`. Соединение выполняется по первичным и внешним ключам, что обеспечивает доступ к строкам за константное время. Поскольку связи изделия с тканью и фурнитурой — многие-ко-многим, к таблицам-связкам `fabric_garment` и `accessories_garment` дополнительно присоединяются сами таблицы `fabric` и `accessories`: это позволяет задавать условие отбора не по числовым идентификаторам, а по имени (артикулу) ткани и фурнитуры. С помощью `WHERE` отбираются только те записи, которые соответствуют заданному ателье (`atelier_id`), заказчику (`client_id`), артикулу ткани (`f.article`) и артикулу фурнитуры (`acc.article`). Для устранения дублирования итоговых строк используется `SELECT DISTINCT`, после чего результат сортируется по фамилии и имени администратора.

Реляционная формула:

$$\pi \begin{pmatrix} \beta(\text{a.admin_id as «id»}), \\ \beta(\text{a.surname as «фамилия»}), \\ \beta(\text{a.name as «имя»}), \\ \beta(\text{a.patronymic as «отчество»}) \end{pmatrix}$$

$$\sigma(\text{a.atelier_id} = 1 \wedge \text{o.client_id} = 212 \wedge \text{f.article} = \text{«ART1»} \wedge \text{acc.article} = \text{«ART2»})$$

$$\text{administrator} \bowtie_{(\text{o.admin_id=a.admin_id})} \text{"order"} \bowtie_{(\text{g.order_id=o.order_id})} \text{garment}$$
$$\bowtie_{(\text{fg.garment_id=g.garment_id})} \text{fabric_garment} \bowtie_{(\text{f.fabric_id=fg.fabric_id})} \text{fabric}$$
$$\bowtie_{(\text{ag.garment_id=g.garment_id})} \text{accessories_garment} \bowtie_{(\text{acc.accessory_id=ag.accessory_id})} \text{accessories}$$

$$\delta \ \ \tau(\text{a.surname, a.name})$$

```
1 SELECT DISTINCT
2     a.admin_id    AS "id",
3     a.surname     AS "фамилия",
4     a.name        AS "имя",
5     a.patronymic  AS "отчество"
```

```
6  FROM administrator   a
7  JOIN "order"         o   ON o.admin_id      = a.admin_id
8  JOIN garment         g   ON g.order_id      = o.order_id
9  JOIN fabric_garment  fg  ON fg.garment_id   = g.garment_id
10 JOIN fabric          f   ON f.fabric_id     = fg.fabric_id
11 JOIN accessories_garment ag  ON ag.garment_id   = g.garment_id
12 JOIN accessories     acc ON acc.accessory_id = ag.accessory_id
13 WHERE a.atelier_id = 1
14   AND o.client_id  = 212
15   AND f.article    = 'TKN-00136'
16   AND acc.article  = 'ACC-00215'
17 ORDER BY a.surname, a.name;
18
```

Листинг 1. Запрос №1.1

В результирующей таблице выведена 1 строка, время выполнения запроса составляет **0,016** секунд. Результат выполнения запроса представлен на рис. 5, представление последовательности выполнения запроса представлено на рис. 6.

```
 id | фамилия |  имя   |  отчество
----+---------+--------+------------
  5 | Никитин | Сергей | Викторович
(1 строка)
```

Рис. 5: Результат запроса №1.1

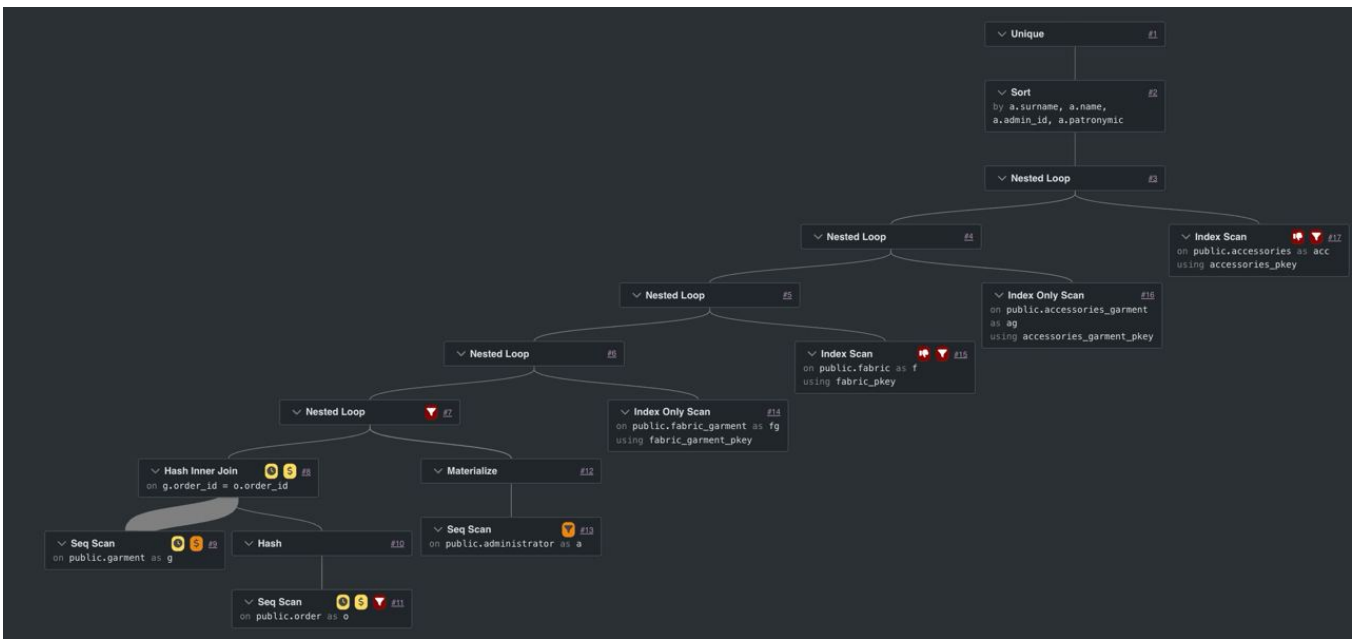


Рис. 6: Представление последовательности выполнения запроса №1.1

5.2 Запрос №1.2

Необходимо найти администраторов, работающих в ателье A, выполнявших заказ для заказчика Б, в котором используется фурнитура В и ткань Г не менее двух раз (то есть число изделий, удовлетворяющих условиям, не менее двух).

Запрос отличается от запроса №1 добавлением группировки `GROUP BY` по полям администратора и фильтра `HAVING COUNT(*) >= 2`, что позволяет отобрать только те записи, в которых совпадение условий встречается дважды и более.

Реляционная формула:

$$\pi \begin{pmatrix} \beta(\text{a.admin_id as «id»}), \\ \beta(\text{a.surname as «фамилия»}), \\ \beta(\text{a.name as «имя»}), \\ \beta(\text{a.patronymic as «отчество»}) \end{pmatrix}$$

$$\sigma(\text{a.atelier_id} = 1 \wedge \text{o.client_id} = 212 \wedge \text{f.article} = \text{«ART1»} \wedge \text{acc.article} = \text{«ART2»})$$

$$\text{administrator} \bowtie_{(\text{o.admin_id}=\text{a.admin_id})} \text{"order"} \bowtie_{(\text{g.order_id}=\text{o.order_id})} \text{garment}$$
$$\bowtie_{(\text{fg.garment_id}=\text{g.garment_id})} \text{fabric_garment} \bowtie_{(\text{f.fabric_id}=\text{fg.fabric_id})} \text{fabric}$$
$$\bowtie_{(\text{ag.garment_id}=\text{g.garment_id})} \text{accessories_garment} \bowtie_{(\text{acc.accessory_id}=\text{ag.accessory_id})} \text{accessories}$$

$$\delta \quad \tau(\text{a.surname, a.name})$$

```sql
SELECT
    a.admin_id    AS "id",
    a.surname     AS "фамилия",
    a.name        AS "имя",
    a.patronymic  AS "отчество",
    COUNT(*)      AS "совпадений"
FROM administrator   a
JOIN "order"         o   ON o.admin_id      = a.admin_id
JOIN garment         g   ON g.order_id      = o.order_id
JOIN fabric_garment  fg  ON fg.garment_id   = g.garment_id
JOIN fabric          f   ON f.fabric_id     = fg.fabric_id
JOIN accessories_garment ag  ON ag.garment_id   = g.garment_id
JOIN accessories     acc ON acc.accessory_id = ag.accessory_id
WHERE a.atelier_id = 1
  AND o.client_id  = 212
  AND f.article    = 'TKN-00136'
  AND acc.article  = 'ACC-00215'
GROUP BY a.admin_id, a.surname, a.name, a.patronymic
HAVING COUNT(*) >= 2
ORDER BY a.surname, a.name;

```

Листинг 2. Запрос №1.2

В результирующей таблице выведена 1 строка (число совпадений — 3), время выполнения запроса составляет **0,012** секунд. Результат выполнения запроса представлен на рис. 7, представление последовательности выполнения запроса представлено на рис. 8.

```
 id | фамилия |  имя   |  отчество  | совпадений
----+---------+--------+------------+------------
  5 | Никитин | Сергей | Викторович |          3
(1 строка)
```

Рис. 7: Результат запроса №1.2

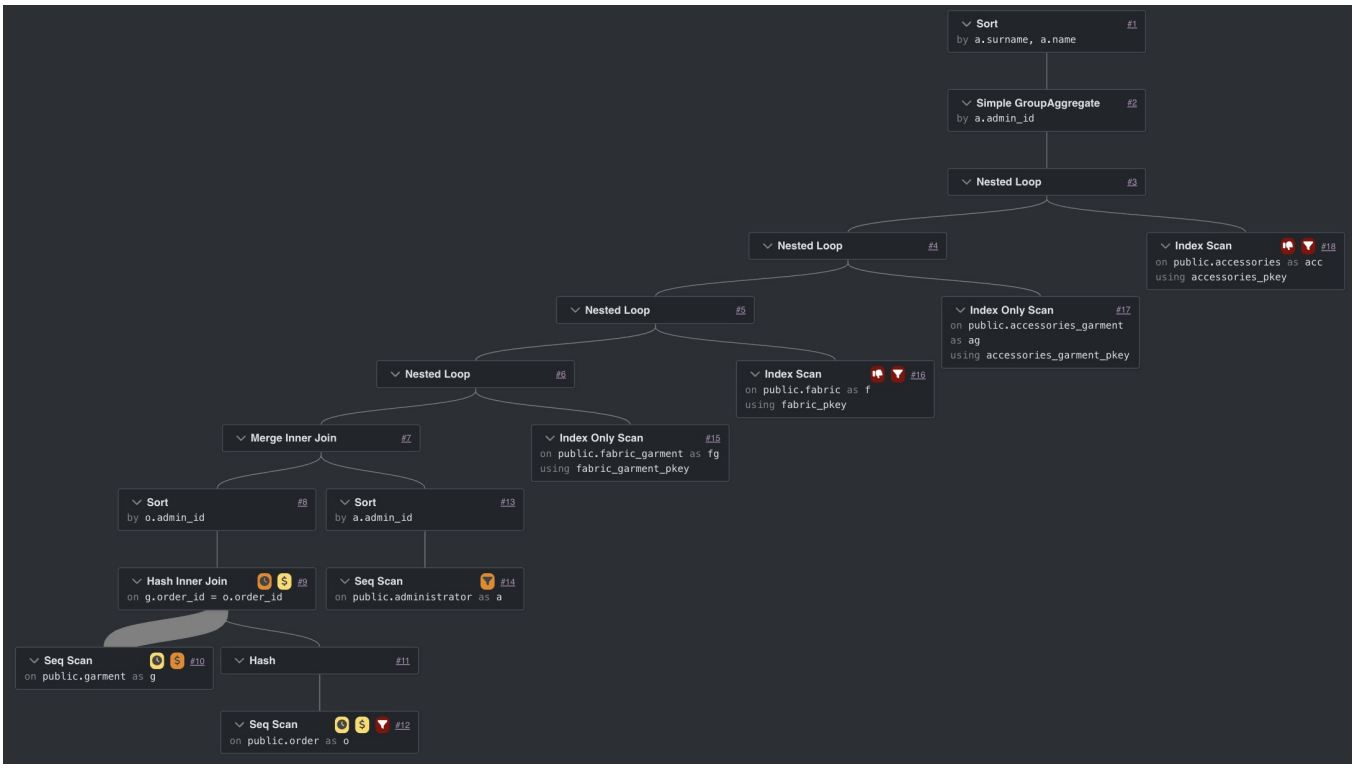


Рис. 8: Представление последовательности выполнения запроса №1.2

5.3 Запрос №2

Необходимо для ателье А рассчитать число изделий, пошитых портным Б.

Текстовое объяснение: таблица `tailor` соединяется с таблицей `garment` по полю `tailor_id`. Условие `WHERE` фильтрует строки по идентификатору ателье и портного, после чего агрегирующая функция `COUNT` подсчитывает число изделий указанного портного. Группировка выполняется по ключевым атрибутам портного.

Реляционная формула:

$$\pi \left(\begin{array}{c} \beta(\text{a.admin_id as «id»}), \\ \beta(\text{a.surname as «фамилия»}), \\ \beta(\text{a.name as «имя»}), \\ \beta(\text{a.patronymic as «отчество»}), \\ \beta(\text{COUNT(*) as «совпадений»}) \end{array} \right)$$

$$\sigma(\text{a.atelier_id} = 1 \wedge \text{o.client_id} = 212 \wedge \text{f.article} = \text{«ART1»} \wedge \text{acc.article} = \text{«ART2»})$$

$$\text{administrator} \bowtie \text{"order"} \bowtie \text{garment}$$

$$\bowtie \text{fabric_garment} \bowtie \text{fabric} \bowtie \text{accessories_garment} \bowtie \text{accessories}$$

$$\gamma_{\text{a.admin_id, a.surname, a.name, a.patronymic}} \left[\text{COUNT(*)} \geq 2 \right]$$

$$\tau(\text{a.surname, a.name})$$

```sql
SELECT
    t.atelier_id        AS "ателье",
    t.tailor_id         AS "портной",
    t.surname           AS "фамилия",
    COUNT(g.garment_id) AS "число изделий"
FROM tailor  t
JOIN garment g ON g.tailor_id = t.tailor_id
WHERE t.atelier_id = 2
  AND t.tailor_id  = 22
GROUP BY t.atelier_id, t.tailor_id, t.surname;
```

Листинг 3. Запрос №2

В результирующей таблице выведена 1 строка (число изделий — 848), время выполнения запроса составляет **0,012** секунд. Результат выполнения запроса представлен на рис. 9, представление последовательности выполнения запроса представлено на рис. 10.

```
 ателье | портной | фамилия | число изделий
--------+---------+---------+---------------
       2 |      22 | Ковалёв |           848
(1 строка)
```

Рис. 9: Результат запроса №2.

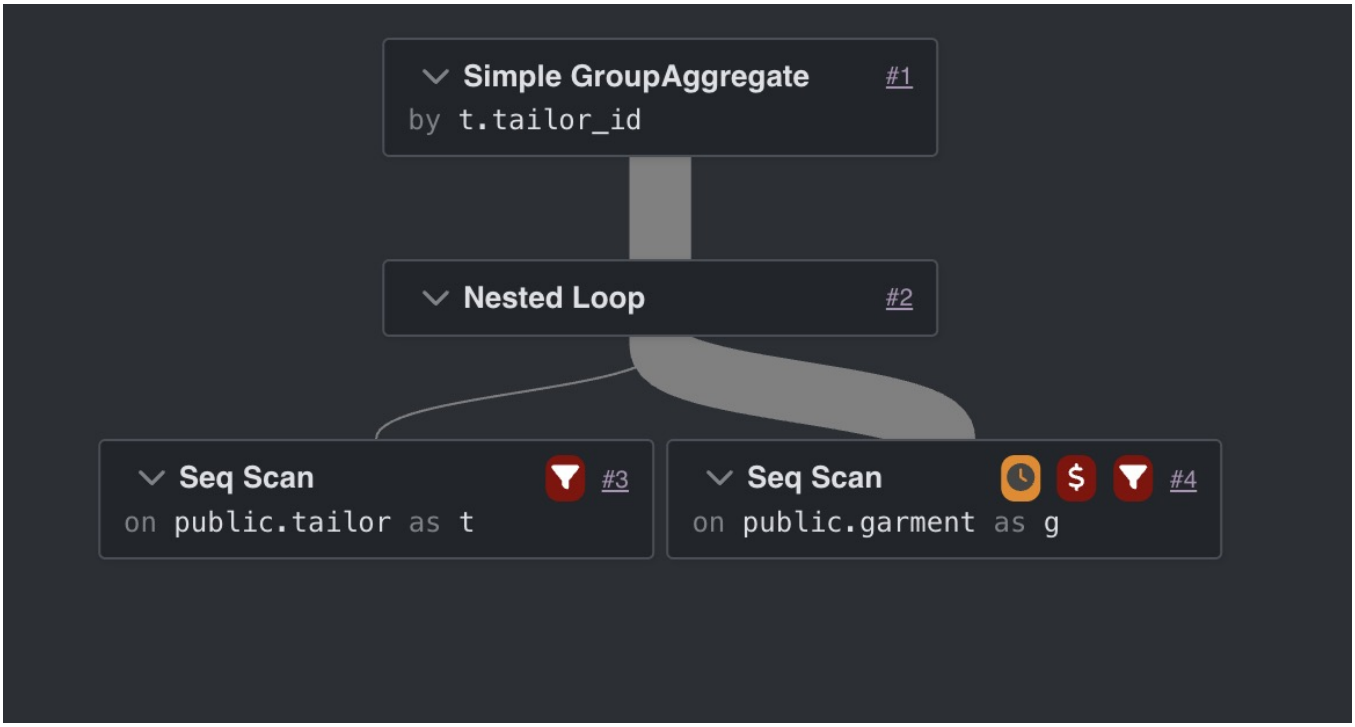


Рис. 10: Представление последовательности выполнения запроса №2

5.4 Запрос №3.1

Необходимо найти заказчика, который заказал максимальное количество фурнитуры по всем ателье.

Текстовое объяснение: соединяются таблицы `client`, `order`, `garment`, `accessories_garment` и `accessories`. Соединение выполняется по первичным и внешним ключам, что обеспечивает доступ к строкам за константное время. Агрегирующая функция `SUM` суммирует значение `quantity_used` по каждому заказчику. Для отбора заказчиков с максимальным количеством фурнитуры используется вложенный подзапрос: во внутренней производной таблице подсчитывается суммарное количество фурнитуры по каждому заказчику, а функция `MAX` выбирает из этих сумм наибольшую. Во внешнем запросе условие `HAVING` оставляет только тех заказчиков, чья сумма равна найденному максимуму. Так как сравнение идёт со значением, а не с конкретным идентификатором, при равенстве максимального количества фурнитуры у нескольких заказчиков в результат попадают все они.

Реляционная формула:

$$\pi \begin{pmatrix} \beta(\text{c.client_id as «id заказчика»}), \\ \beta(\text{c.surname as «фамилия»}), \\ \beta(\text{c.name as «имя»}), \\ \beta(\text{c.patronymic as «отчество»}), \\ \beta(\text{SUM(acc.quantity_used) as «количество фурнитуры»}) \end{pmatrix}$$

$$\text{client} \bowtie_{(\text{o.client_id}=\text{c.client_id})} \text{"order"} \bowtie_{(\text{g.order_id}=\text{o.order_id})} \text{garment} \bowtie_{(\text{ag.garment_id}=\text{g.garment_id})} \text{accessories_garment} \bowtie_{(\text{acc.accessory_id}=\text{ag.accessory_id})} \text{accessories}$$

$$\gamma_{\text{c.client_id, c.surname, c.name, c.patronymic}}\left[\,\text{SUM(acc.quantity_used)} = M\,\right]$$

$$\text{где } M = \max\Big(\gamma_{\text{o2.client_id}}\big[\text{SUM(acc2.quantity_used)}\big]\big(\text{"order"} \bowtie \text{garment} \bowtie$$

$$\text{accessories_garment} \bowtie \text{accessories}\big)\Big)$$

$$\tau(\text{c.client_id})$$

```sql
SELECT
    c.client_id     AS "id заказчика",
    c.surname       AS "фамилия",
    c.name          AS "имя",
    c.patronymic    AS "отчество",
    SUM(acc.quantity_used) AS "количество фурнитуры"
FROM client      c
JOIN "order"     o  ON o.client_id    = c.client_id
JOIN garment     g  ON g.order_id     = o.order_id
JOIN accessories_garment ag ON ag.garment_id = g.garment_id
JOIN accessories acc ON acc.accessory_id = ag.accessory_id
GROUP BY c.client_id, c.surname, c.name, c.patronymic
HAVING SUM(acc.quantity_used) = (
    SELECT MAX(t.total)
    FROM (
        SELECT SUM(acc2.quantity_used) AS total
        FROM "order"     o2
        JOIN garment     g2  ON g2.order_id     = o2.order_id
        JOIN accessories_garment ag2 ON ag2.garment_id = g2.garment_id
        JOIN accessories acc2 ON acc2.accessory_id = ag2.accessory_id
        GROUP BY o2.client_id
    ) t
)
ORDER BY c.client_id;
```

Листинг 4. Запрос №3.1

В результирующей таблице выведено 2 строки: максимальное суммарное количество фурнитуры (по 70 000 единиц) одновременно достигнуто двумя заказчиками, и оба попадают в результат, так как условие `HAVING` сравнивает их сумму со значением максимума, найденным во вложенном подзапросе. Время выполнения запроса составляет **0,311** секунд. Результат выполнения запроса представлен на рис. 11, представление последовательности выполнения запроса представлено на рис. 12.

```
 id заказчика | фамилия |  имя   | отчество | количество фурнитуры
--------------+---------+--------+----------+----------------------
         2003 | Тестов  | Первый | Тестович |                70000
         2004 | Тестов  | Второй | Тестович |                70000
(2 строки)
```

Рис. 11: Результат запроса №3.1

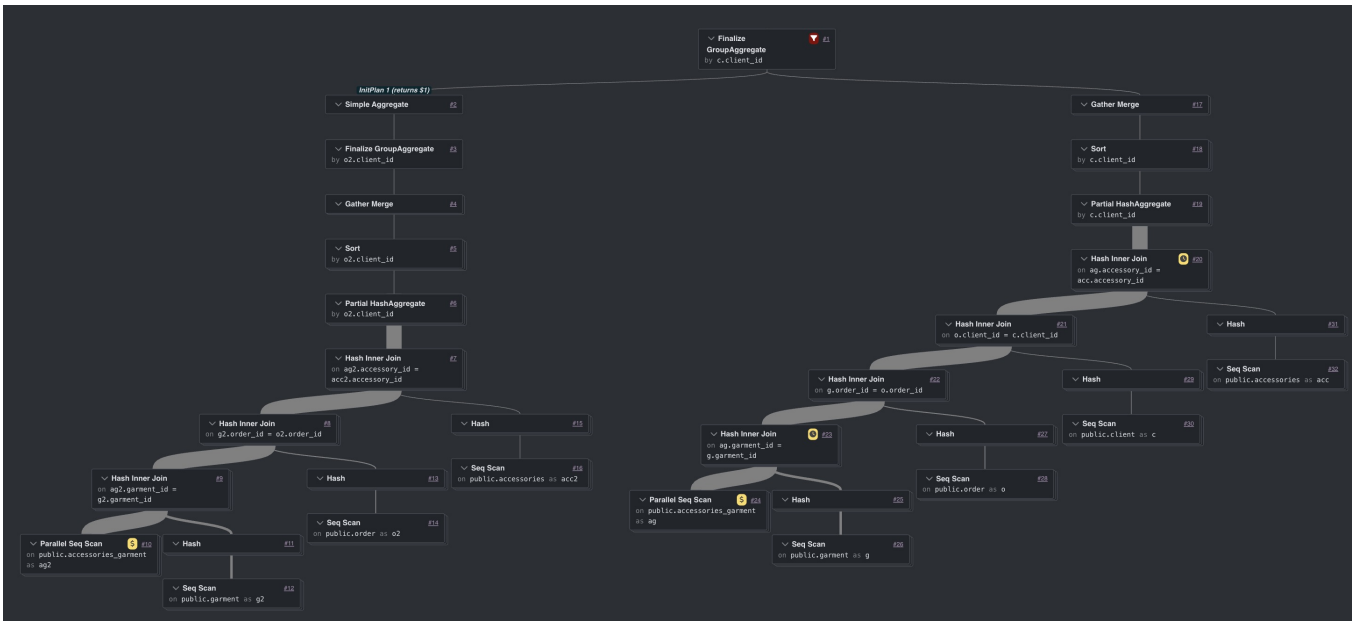


Рис. 12: Представление последовательности выполнения запроса №3.1

5.5 Запрос №3.2

Необходимо найти заказчика, который заказал минимальное количество фурнитуры по всем ателье.

Запрос отличается от запроса №3.1 только агрегирующей функцией во вложенном подзапросе: вместо `MAX` используется `MIN`, выбирающая наименьшую из сумм количества фурнитуры по заказчикам. Во внешнем запросе условие `HAVING` оставляет тех заказчиков, чья сумма равна найденному минимуму, поэтому при равенстве минимального количества фурнитуры у нескольких заказчиков в результат попадают все они.

Реляционная формула:

$$\pi \begin{pmatrix} \beta(\text{c.client_id as «id заказчика»}), \\ \beta(\text{c.surname as «фамилия»}), \\ \beta(\text{c.name as «имя»}), \\ \beta(\text{c.patronymic as «отчество»}), \\ \beta(\text{SUM(acc.quantity_used) as «количество фурнитуры»}) \end{pmatrix}$$

$$\text{client} \bowtie_{(\text{o.client_id}=\text{c.client_id})} \text{"order"} \bowtie_{(\text{g.order_id}=\text{o.order_id})} \text{garment}$$

$$\bowtie_{(\text{ag.garment_id}=\text{g.garment_id})} \text{accessories_garment} \bowtie_{(\text{acc.accessory_id}=\text{ag.accessory_id})} \text{accessories}$$

$$\gamma_{\text{c.client_id, c.surname, c.name, c.patronymic}} \left[\text{SUM(acc.quantity_used)} = m \right]$$

$$\text{где } m = \min\left(\gamma_{\text{o2.client_id}}\left[\text{SUM(acc2.quantity_used)}\right]\left(\text{"order"} \bowtie \text{garment} \bowtie \right.\right.$$

$$\left.\left. \bowtie \text{accessories_garment} \bowtie \text{accessories}\right)\right)$$

$$\tau(\text{c.client_id})$$

```sql
SELECT
    c.client_id     AS "id заказчика",
    c.surname       AS "фамилия",
    c.name          AS "имя",
    c.patronymic    AS "отчество",
    SUM(acc.quantity_used) AS "количество фурнитуры"
FROM client      c
JOIN "order"     o  ON o.client_id    = c.client_id
JOIN garment     g  ON g.order_id     = o.order_id
JOIN accessories_garment ag ON ag.garment_id = g.garment_id
JOIN accessories acc ON acc.accessory_id = ag.accessory_id
GROUP BY c.client_id, c.surname, c.name, c.patronymic
HAVING SUM(acc.quantity_used) = (
    SELECT MIN(t.total)
    FROM (
        SELECT SUM(acc2.quantity_used) AS total
        FROM "order"     o2
        JOIN garment     g2  ON g2.order_id     = o2.order_id
        JOIN accessories_garment ag2 ON ag2.garment_id = g2.garment_id
        JOIN accessories acc2 ON acc2.accessory_id = ag2.accessory_id
        GROUP BY o2.client_id
    ) t
)
ORDER BY c.client_id;
```

Листинг 5. Запрос №3.2

В результирующей таблице выведено 2 строки: минимальное суммарное количество фурнитуры (по 1 единице) одновременно достигнуто двумя заказчиками, и оба попадают в результат, так как условие `HAVING` сравнивает их сумму со значением минимума, найденным во вложенном подзапросе. Время выполнения запроса составляет **0,280** секунд. Результат выполнения запроса представлен на рис. 13, представление последовательности выполнения запроса представлено на рис. 14.

```
 id заказчика | фамилия |  имя   | отчество | количество фурнитуры
--------------+---------+--------+----------+----------------------
         2005 | Минимов | Первый | Тестович |                    1
         2006 | Минимов | Второй | Тестович |                    1
(2 строки)
```

Рис. 13: Результат запроса №3.2.

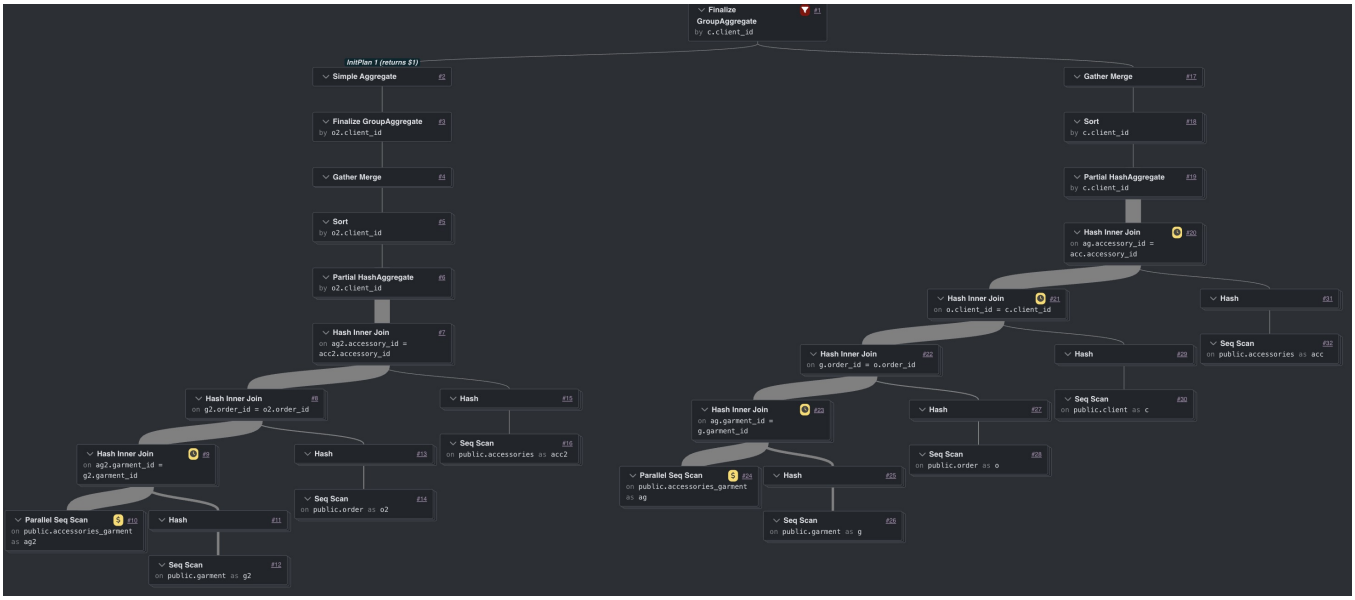


Рис. 14: Представление последовательности выполнения запроса №3.2

5.6 Запрос №4

Необходимо посчитать число заказчиков с одинаковым числом изделий в их заказах.

Во вложенной (производной) таблице происходит группировка по заказчику `client_id` и подсчёт числа связанных с ним изделий при помощи `COUNT`. Соединение таблиц `"order"` и `garment` выполняется по внешнему ключу `order_id`. Во внешнем запросе результаты группируются уже по числу изделий `garments_per_client`, и подсчитывается, сколько заказчиков приходится на каждое такое значение.

Реляционная формула:

$$\pi \begin{pmatrix} \beta(\text{garments_per_client as «число изделий»}), \\ \beta(\text{COUNT}(*) \text{ as «число заказчиков»}) \end{pmatrix}$$

$$\gamma(\text{garments_per_client})$$

$$\tau(\text{garments_per_client})$$

$$\text{где garments_per_client}$$

$$\pi \begin{pmatrix} \beta(\text{o.client_id}), \\ \beta(\text{COUNT(g.garment_id) as garments_per_client}) \end{pmatrix}$$

$$\text{"order"} \bowtie_{(\text{g.order_id}=\text{o.order_id})} \text{garment}$$

$$\gamma(\text{o.client_id})$$

```sql
SELECT
    t.garments_per_client  AS "число изделий",
    COUNT(*)               AS "число заказчиков"
FROM (
    SELECT
        o.client_id,
        COUNT(g.garment_id) AS garments_per_client
    FROM "order" o
    JOIN garment g ON g.order_id = o.order_id
    GROUP BY o.client_id
) AS t
GROUP BY t.garments_per_client
ORDER BY t.garments_per_client;
```

Листинг 5. Запрос №4

В результирующей таблице выведено 96 строк, время выполнения запроса составляет **0,041** секунд. Результат выполнения запроса представлен на рис. 15, представление последовательности выполнения запроса представлено на рис. 16. Двумерная диаграмма, демонстрирующая распределение числа изделий среди заказчиков представлена на рис. 17.

число изделий	число заказчиков
4	1
6	1
7	2
8	1
9	4
10	3
11	4
12	1
13	7
14	8
15	7
16	5
17	16
18	4
19	10
20	8
21	15
22	12
23	8
24	10
25	12
26	12
27	13
28	12
29	11
30	13
31	13
32	22
33	16
34	19
35	14
36	16
37	21
38	18

Рис. 15: Результат запроса №4.

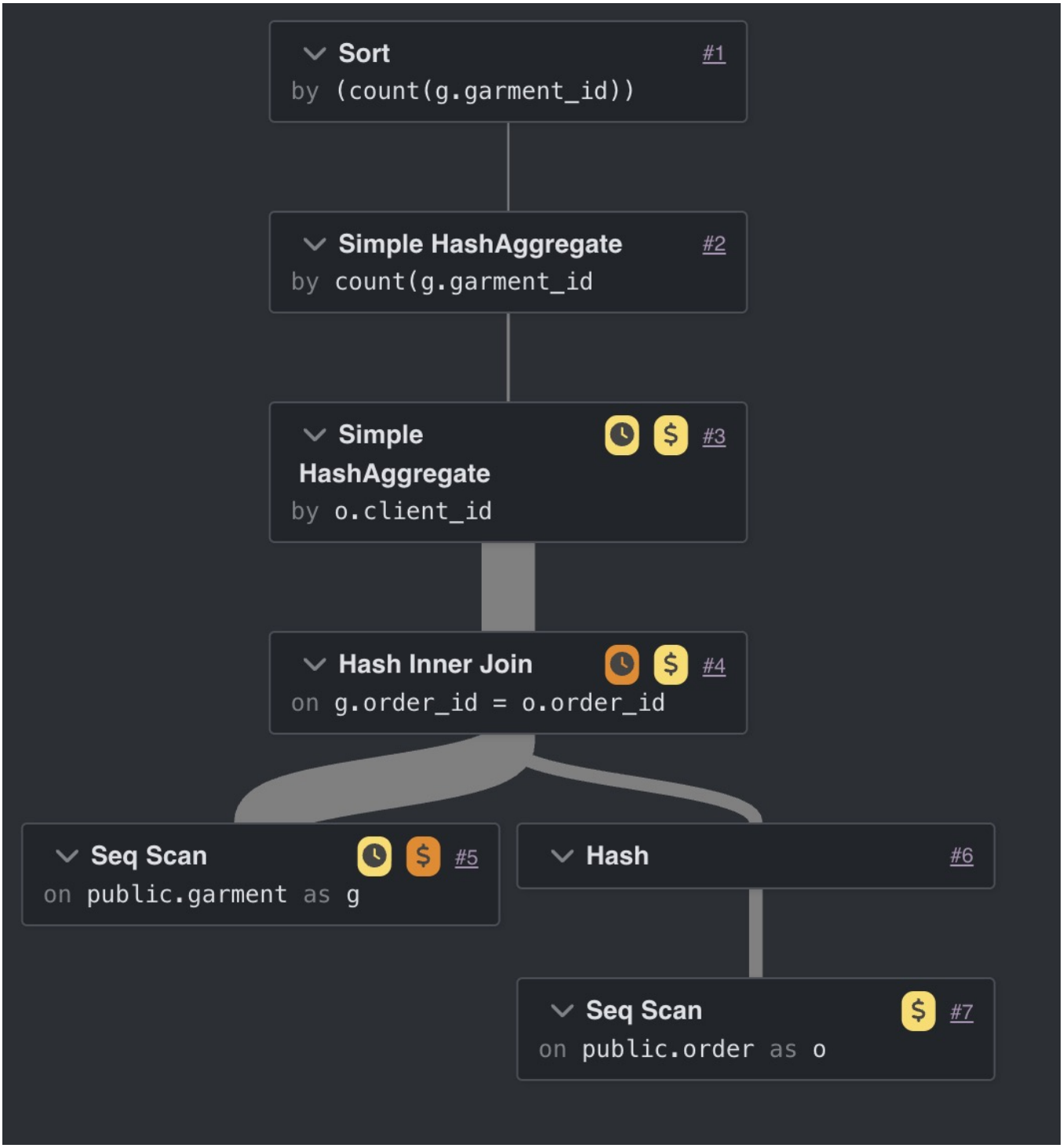


Рис. 16: Представление последовательности выполнения запроса №4



Рис. 17: Двумерная гистограмма распределения числа изделий на заказчика.

5.7 Запрос №5

Необходимо для каждого заказчика посчитать число заказов и число изделий. Результат используется для построения двумерной гистограммы.

Текстовое объяснение: таблица `client` соединяется с `order` и `garment` по внешним ключам. Группировка выполняется по идентификатору заказчика. Для подсчёта заказов используется `COUNT(DISTINCT o.order_id)` (так как один заказ соответствует нескольким изделиям и без `DISTINCT` строки дублировались бы), для подсчёта изделий — `COUNT(g.garment_id)`.

Реляционная формула:

$$\pi \left(\begin{array}{c} \beta(\text{c.client_id as «id»}), \\ \beta(\text{c.surname as «фамилия»}), \\ \beta(\text{c.name as «имя»}), \\ \beta(\text{COUNT(DISTINCT o.order_id) as «заказов»}), \\ \beta(\text{COUNT(g.garment_id) as «изделий»}) \end{array} \right)$$

$$\text{client} \bowtie \text{"order"} \bowtie \text{garment}$$

$$\gamma(\text{c.client_id, c.surname, c.name})$$

```sql
SELECT
    c.client_id                  AS "id",
    c.surname                    AS "фамилия",
    c.name                       AS "имя",
    COUNT(DISTINCT o.order_id)   AS "заказов",
    COUNT(g.garment_id)          AS "изделий"
FROM client  c
JOIN "order" o ON o.client_id = c.client_id
JOIN garment g ON g.order_id  = o.order_id
GROUP BY c.client_id, c.surname, c.name
ORDER BY c.client_id;
```

Листинг 7. Запрос №5

В результирующей таблице выведено 1 000 строк, время выполнения запроса составляет **0,081** секунд. Результат выполнения запроса представлен на рис. 18, представление последовательности выполнения запроса представлено на рис. 19. Двумерная гистограмма зависимости числа изделий от числа заказов представлена на рис. 20.

id	фамилия	имя	заказов	изделий
1	Иванов	Михаил	15	72
2	Герасимов	Владимир	6	25
3	Новикова	Наталья	6	23
4	Семёнова	Анна	5	35
5	Тихонова	Наталья	3	16
6	Борисов	Константин	10	32
7	Семёнова	Ольга	12	66
8	Никитин	Константин	20	95
9	Попова	Мария	12	70
10	Попова	Людмила	7	28
11	Зубова	Валентина	7	19
12	Борисов	Артём	3	15
13	Романов	Владимир	9	56
14	Тихонов	Алексей	9	41
15	Сидорова	Людмила	7	31
16	Тихонов	Николай	11	34
17	Виноградова	Светлана	6	37
18	Зубова	Ольга	8	38
19	Ковалёв	Алексей	4	19
20	Ковалёв	Артём	10	45

Рис. 18: Фрагмент результата запроса №5.

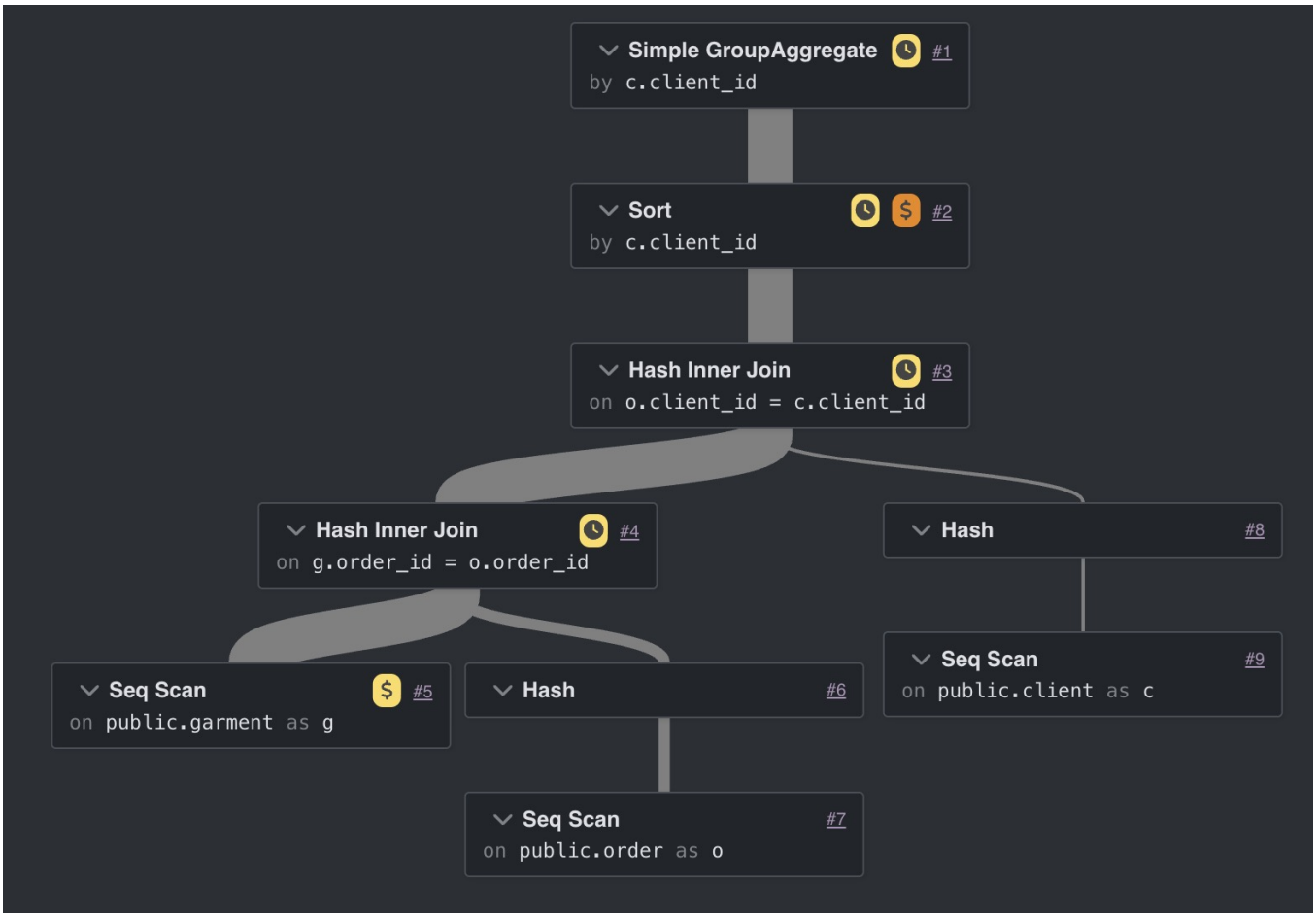


Рис. 19: Представление последовательности выполнения запроса №5

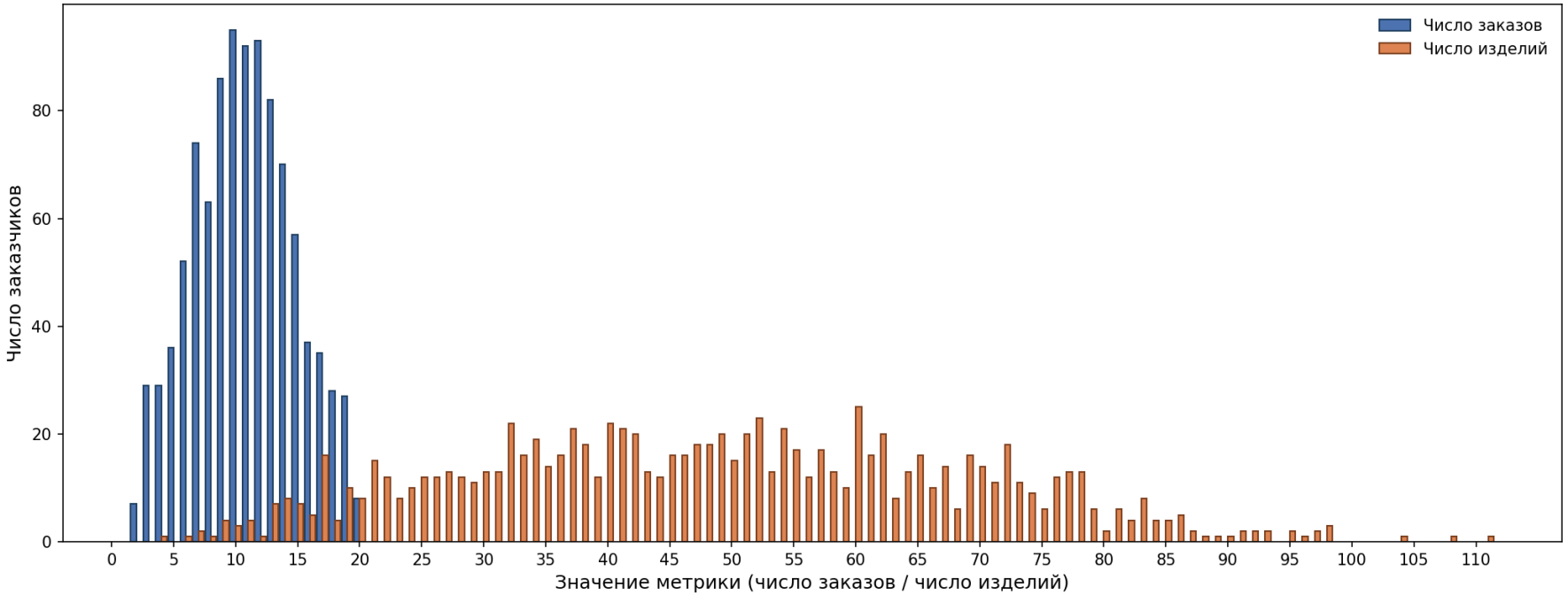


Рис. 20: Двумерная гистограмма распределения числа изделий и числа заказов по заказчикам.

5.8 Запрос №6

Необходимо найти типы фурнитуры, которые использовались чаще, чем фурнитура A.

Текстовое объяснение: таблица `accessories` соединяется с `accessories_garment` по `accessory_id`. Затем выполняется группировка по типу фурнитуры (`type`) с подсчётом числа использований. В `HAVING` используется скалярный подзапрос, возвращающий число использований конкретной фурнитуры A — это позволяет одной строкой задать порог отсечения, не строя при этом многоуровневую вложенность.

Реляционная формула: пусть

$$n_A = |\sigma_{\text{accessory_id}=A}(\text{accessories_garment})|$$

— число использований фурнитуры A. Тогда

$$\pi \begin{pmatrix} \beta(\text{a.type as «тип фурнитуры»}), \\ \beta(\text{COUNT(ag.garment_id) as «использований»}) \end{pmatrix}$$

$$\text{accessories} \bowtie \text{accessories_garment}$$

$$\gamma(\text{a.type}),\ \sigma(\text{COUNT(ag.garment_id)} > n_A),\ \tau_{\downarrow}(\text{использований}).$$

```sql
SELECT
    a.type                      AS "тип фурнитуры",
    COUNT(ag.garment_id)        AS "использований"
FROM accessories a
JOIN accessories_garment ag ON ag.accessory_id = a.accessory_id
GROUP BY a.type
HAVING COUNT(ag.garment_id) > (
    SELECT COUNT(*)
    FROM accessories_garment
    WHERE accessory_id = 296
)
ORDER BY "использований" DESC;
```

Листинг 8. Запрос №6

В результирующей таблице выведено 10 строк (для выбранной фурнитуры A все 10 типов оказались использованы чаще, что объясняется тем, что каждый тип агрегирует около 100 различных фурнитур), время выполнения запроса составляет **0,215** секунд. Результат выполнения запроса представлен на рис. 21, представление последовательности выполнения запроса представлено на рис. 22.

тип фурнитуры	использований
Кнопка	144430
Молния	127680
Шлёвка	127606
Застёжка	116570
Липучка	112387
Крючок	107903
Заклёпка	105669
Пуговица	103373
Петля	96845
Бретель	81131

(10 строк)

Рис. 21: Результат запроса №6.

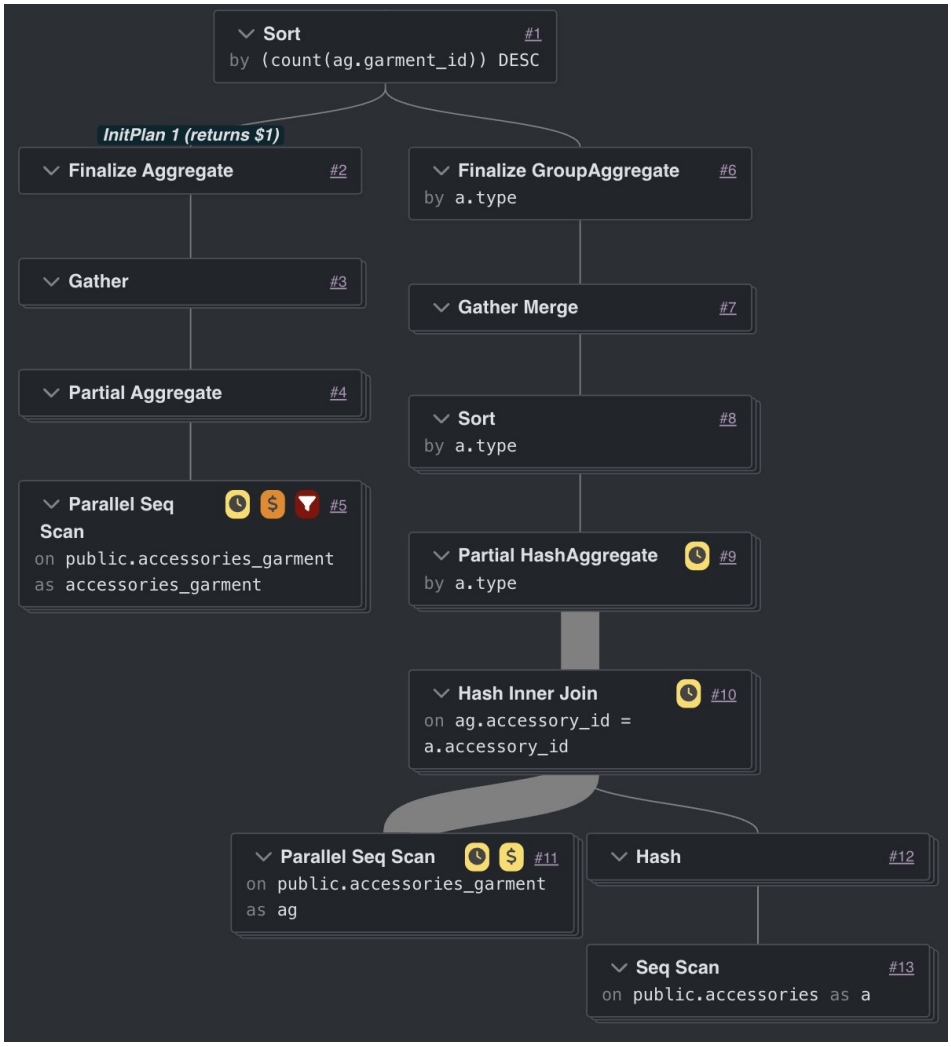


Рис. 22: Представление последовательности выполнения запроса №6

5.9 Запрос №7

Необходимо найти всех заказчиков, которые ни разу не использовали ткань A.

Текстовое объяснение: применяется техника «антисоединения» через `LEFT JOIN` с фильтром, перенесённым в условие `ON` (а не в `WHERE`). Для каждого заказчика оставляются только те связи изделия с тканью A, которые существуют; после группировки по заказчику выбираются те, у которых число таких связей равно нулю. Это позволяет реализовать поиск «не существует» без использования вложенного подзапроса с `NOT IN` или `NOT EXISTS`.

Реляционная формула:

$$\pi \begin{pmatrix} \beta(\text{c.client_id as «id»}), \\ \beta(\text{c.surname as «фамилия»}), \\ \beta(\text{c.name as «имя»}), \\ \beta(\text{c.patronymic as «отчество»}) \end{pmatrix}$$

$$\text{client} \bowtie \text{"order"} \bowtie \text{garment} \bowtie \sigma_{\text{fabric_id}=A}(\text{fabric_garment})$$

$$\gamma(\text{c.client_id, c.surname, c.name, c.patronymic}),$$

$$\sigma(\text{COUNT(fg.fabric_id)} = 0),\ \tau(\text{c.surname, c.name}).$$

Здесь $\bowtie$ обозначает левое внешнее соединение.

```sql
SELECT
    c.client_id   AS "id",
    c.surname     AS "фамилия",
    c.name        AS "имя",
    c.patronymic  AS "отчество"
FROM client          c
LEFT JOIN "order"    o  ON o.client_id  = c.client_id
LEFT JOIN garment    g  ON g.order_id   = o.order_id
LEFT JOIN fabric_garment fg
       ON fg.garment_id = g.garment_id
      AND fg.fabric_id  = 495
GROUP BY c.client_id, c.surname, c.name, c.patronymic
HAVING COUNT(fg.fabric_id) = 0
ORDER BY c.surname, c.name;
```

Листинг 9. Запрос №7

В результирующей таблице выведено 1 657 строк, время выполнения запроса составляет **0,065** секунд. Результат выполнения запроса представлен на рис. 23, представление последовательности выполнения запроса представлено на рис. 24.

id	фамилия	имя	отчество
1998	Беляев	Алексей	Дмитриевна
663	Беляев	Алексей	Александровна
1647	Беляев	Алексей	Ивановна
50	Беляев	Андрей	
1272	Беляев	Артём	
1285	Беляев	Артём	
1213	Беляев	Артём	Дмитриевна
1497	Беляев	Владимир	Сергеевна
196	Беляев	Владимир	Сергеевна
814	Беляев	Владимир	Ивановна
1986	Беляев	Геннадий	Геннадьевич
1679	Беляев	Геннадий	Александрович
1829	Беляев	Геннадий	
26	Беляев	Геннадий	Викторович
735	Беляев	Дмитрий	Николаевна
52	Беляев	Дмитрий	Николаевна

Рис. 23: Фрагмент результата запроса №7.

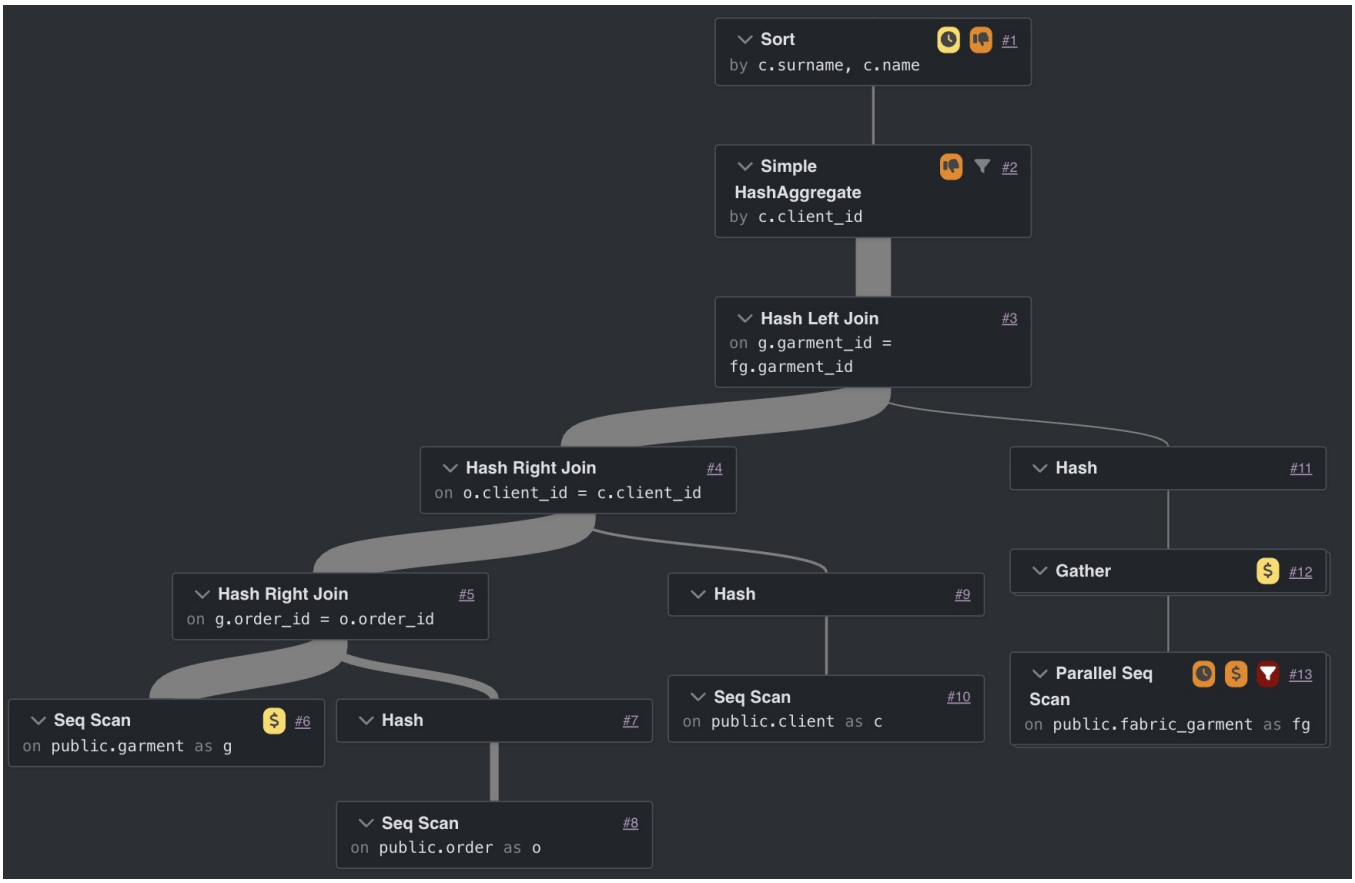


Рис. 24: Представление последовательности выполнения запроса №7

5.10 Запрос №8

Необходимо для каждого ателье и каждого администратора посчитать число заказов. Результат используется для построения трёхмерной гистограммы.

Текстовое объяснение: таблицы `atelier` и `administrator` соединяются единственным декартовым произведением `CROSS JOIN`, а условие связи администратора с рассматриваемым ателье (`ad.atelier_id = at.atelier_id`) вынесено в раздел `WHERE`. Заказы присоединяются отдельно через левое внешнее соединение `LEFT JOIN` по условию `o.admin_id = ad.admin_id`. Использование `LEFT JOIN` важно: если у администратора нет ни одного заказа, он всё равно попадает в результат с нулевым значением `COUNT`, тогда как при обычном внутреннем соединении такие пары были бы потеряны. Группировка выполняется по идентификатору ателье и администратора, а функция `COUNT` подсчитывает число заказов. По осям трёхмерной гистограммы откладываются `atelier_id`, `admin_id` и число заказов.

Реляционная формула:

$$\pi \begin{pmatrix} \beta(\text{at.atelier_id as «ателье»}), \\ \beta(\text{ad.admin_id as «админ»}), \\ \beta(\text{ad.surname as «фамилия»}), \\ \beta(\text{COUNT(o.order_id) as «число заказов»}) \end{pmatrix}$$

$$\sigma_{(\text{ad.atelier_id}=\text{at.atelier_id})}\big(\text{atelier} \times \text{administrator}\big) \bowtie_{(\text{o.admin_id}=\text{ad.admin_id})} \text{"order"}$$

$$\gamma_{\text{at.atelier_id},\,\text{ad.admin_id},\,\text{ad.surname}}\big[\text{COUNT(o.order_id)}\big]$$

$$\tau(\text{at.atelier_id},\ \text{ad.admin_id})$$

```sql
SELECT
    at.atelier_id     AS "ателье",
    ad.admin_id       AS "админ",
    ad.surname        AS "фамилия",
    COUNT(o.order_id) AS "заказов"
FROM atelier        at
CROSS JOIN administrator  ad
LEFT JOIN "order"   o  ON o.admin_id = ad.admin_id
WHERE ad.atelier_id = at.atelier_id
GROUP BY at.atelier_id, ad.admin_id, ad.surname
ORDER BY at.atelier_id, ad.admin_id;
```

Листинг 10. Запрос №8

В результирующей таблице выведено 86 строк (по числу администраторов), время выполнения запроса составляет **0,012** секунд. Результат выполнения запроса представлен на рис. 25, представление последовательности выполнения запроса представлено на рис. 26. Трёхмерная гистограмма распределения числа заказов по ателье и администраторам представлена на рисунке 27.

ателье	админ	фамилия	заказов
1	1	Ковалёва	283
1	2	Никитин	298
1	3	Виноградова	312
1	4	Волкова	262
1	5	Никитин	264
1	29	Морозова	58
1	30	Семёнов	65
1	31	Михайлов	71
1	32	Соколова	71
1	33	Сидорова	79
1	34	Борисова	54
1	35	Иванов	53
2	6	Макарова	327
2	7	Зайцева	299
2	8	Петров	282

Рис. 25: Фрагмент результата запроса №8.

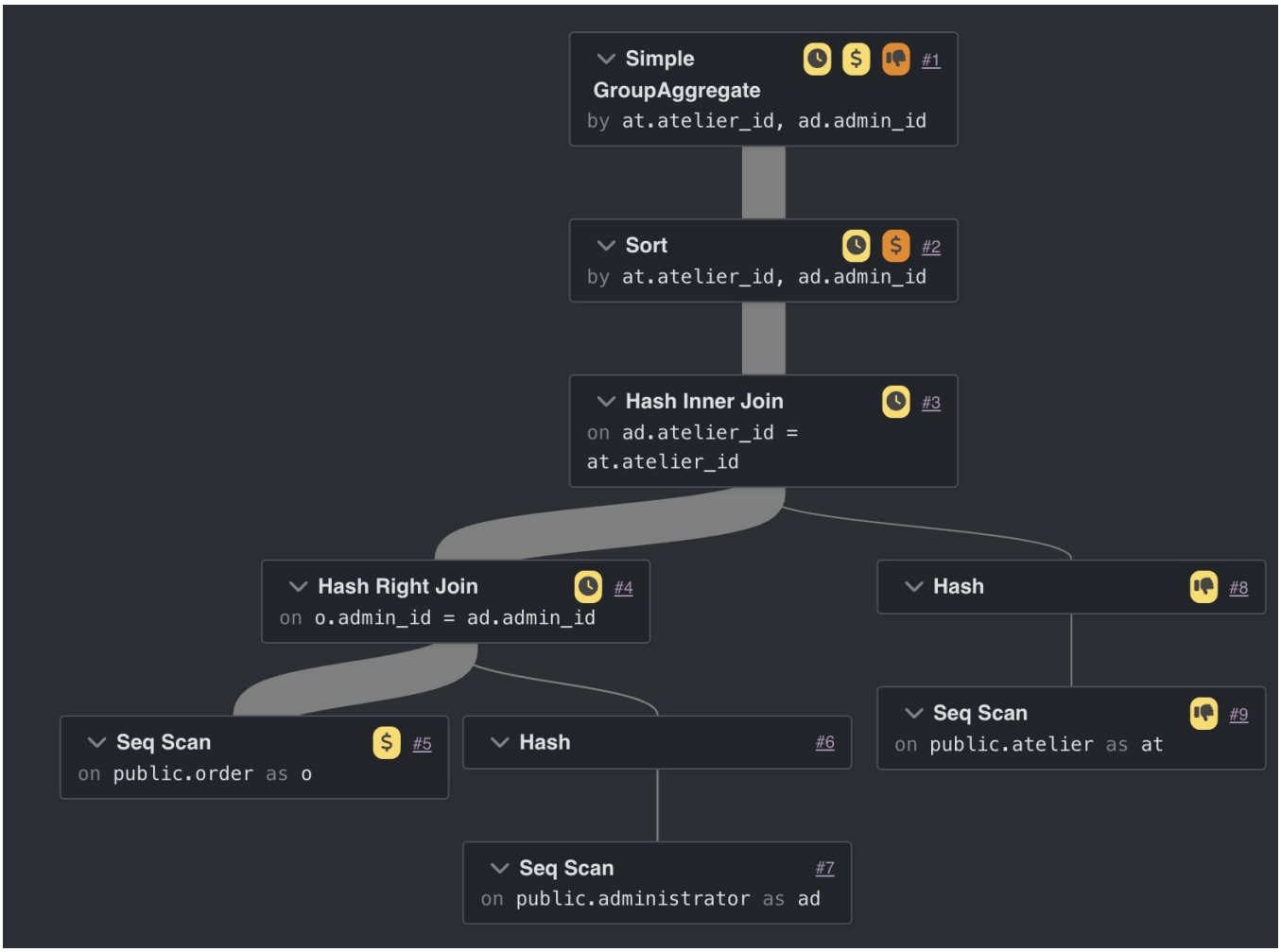


Рис. 26: Представление последовательности выполнения запроса №8

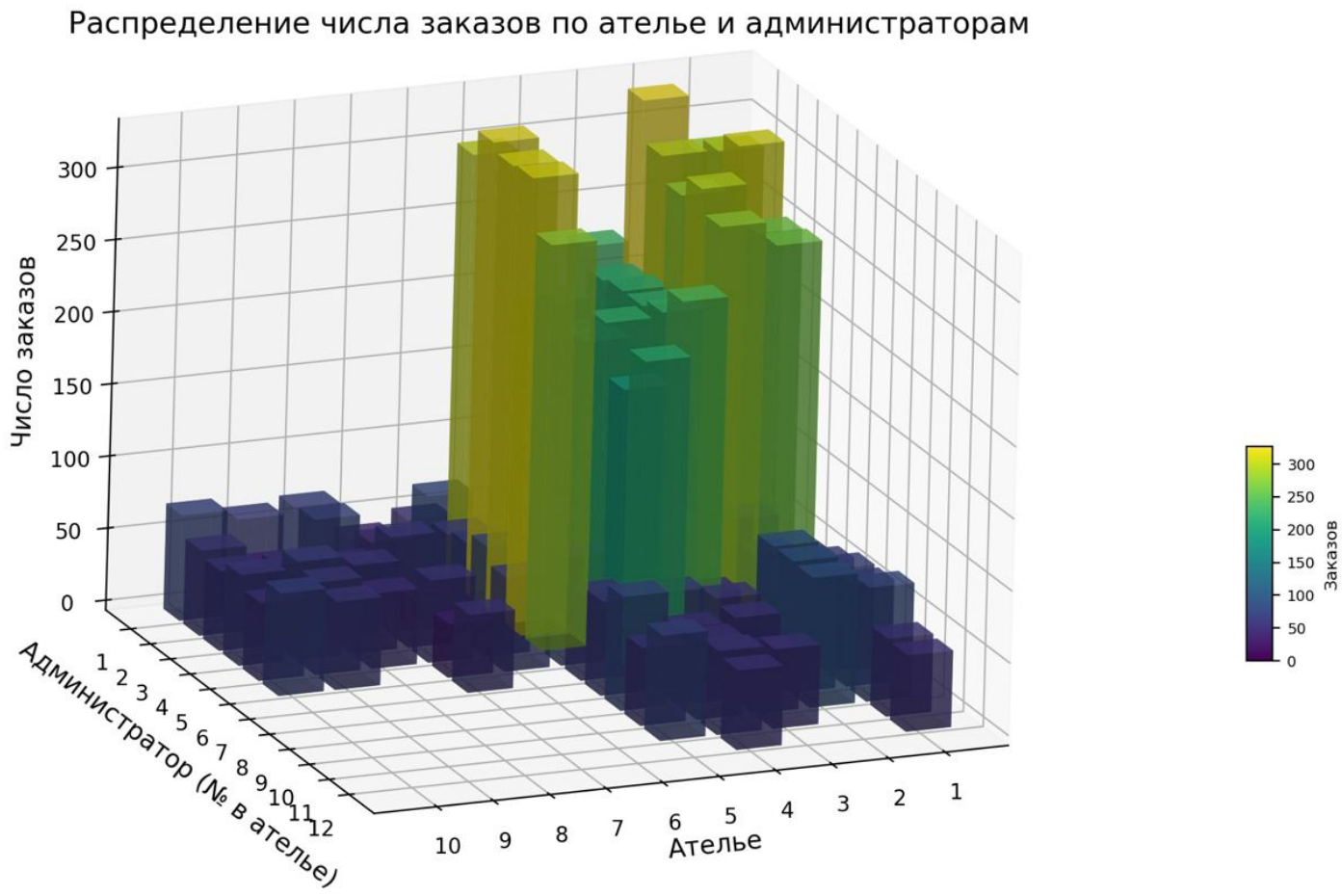


Рис. 27: Трёхмерная гистограмма распределения числа заказов по ателье и администраторам.

5.11 Запрос №9

Необходимо для изделия А заменить фурнитуру А на фурнитуру Б.

Текстовое объяснение: запрос выполняется командой `UPDATE` для таблицы `accessories_garment`. Условие `WHERE` отбирает строку, соответствующую конкретной паре <изделие, фурнитура А>, после чего значение `accessory_id` заменяется на идентификатор фурнитуры Б. Так как пара `(garment_id, accessory_id)` образует составной первичный ключ, перед выполнением необходимо убедиться, что у указанного изделия не существует связи с фурнитурой Б — иначе будет нарушено ограничение уникальности.

```
1 UPDATE accessories_garment
2 SET accessory_id = 200
3 WHERE garment_id   = 6
4   AND accessory_id = 100;
```

Листинг 11. Запрос №9

Команда `UPDATE` затронула 1 строку, время выполнения составляет **0,008** секунд. Результат выполнения запроса представлен на рис. 28, представление последовательности выполнения запроса представлено на рис. 29.

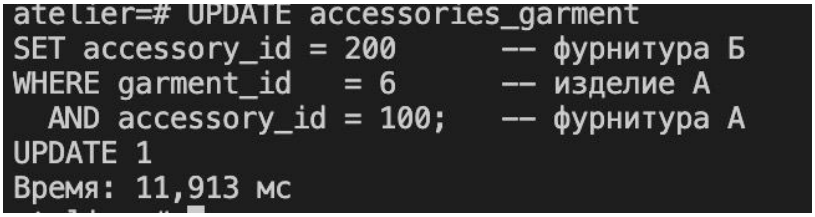


Рис. 28: Результат запроса №9.

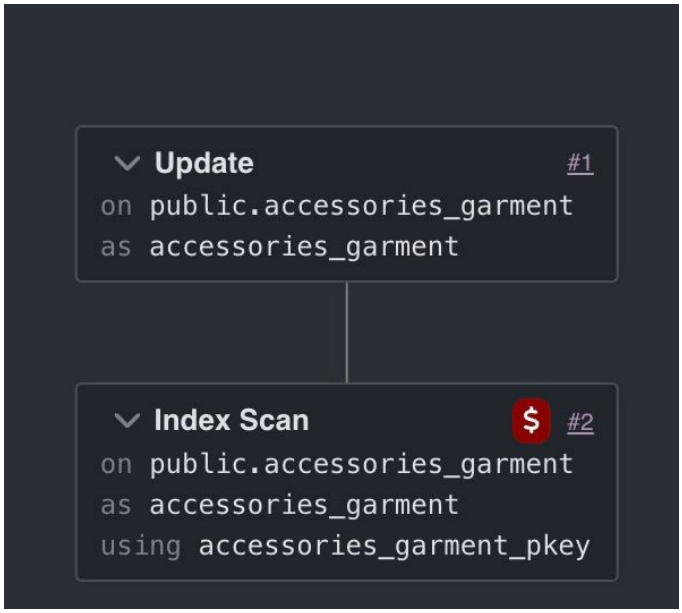


Рис. 29: Представление последовательности выполнения запроса №9

Заключение

В ходе работы выполнен анализ предметной области «Учет заказов ателье индивидуального пошива», определены цели функционирования базы данных, выявлены сущности и их атрибуты. На основе анализа построены ER-диаграмма и схема базы данных, описаны таблицы с обоснованием типов атрибутов.

Схема реализована в СУБД PostgreSQL. Для заполнения базы данных написана программа-генератор на языке C++ с использованием библиотеки **libpq**. База данных содержит 11 таблиц, в которых хранится 1 083 301 запись.

Выполнены 9 запросов к базе данных. Для каждого запроса приведены результат выполнения запроса, графический план выполнения, текстовое описание и реляционная формула.

Работа выполнялась на устройстве Apple MacBook с процессором Apple с операционной системой macOS; в качестве СУБД использовался PostgreSQL. Визуализация результатов запросов выполнена средствами Python (**matplotlib**).

Список литературы

[1] Попов С.Г. Лист самостоятельной проверки хода выполнения курсовой работы по дисциплине «Теоретические основы баз данных». 4 семестр.

[2] PostgreSQL 15 Documentation [Электронный ресурс]: The PostgreSQL Global Development Group. URL: https://www.postgresql.org/docs/15 (дата обращения: 22.05.2026).

[3] Такахаси М., Адзума С. The Manga Guide to Databases. — San Francisco: No Starch Press, 2009. — 224 с.

[4] Электронный каталог тканей Loro Piana [Электронный ресурс]: Бюро тканей. URL: https://burotkani.com/catalog/loro-piana/ (дата обращения: 22.05.2026).

[5] Как работает ателье по пошиву одежды? [Электронный ресурс]: Фабрика пошива. URL: https://fabrika-poshiva.su/stati/kak-rabotaet-atele-po-poshivu-odezhdy.html (дата обращения: 22.05.2026).

6 Приложение А: Программа генерации базы данных

```sql
SET client_encoding = 'UTF8';

-- 1. ENUM-типы
CREATE TYPE order_status AS ENUM (
    'Принят',
    'В работе',
    'Готов',
    'Выдан',
    'Отменён'
);

CREATE TYPE garment_type AS ENUM (
    'Пальто',
    'Пиджак',
    'Брюки',
    'Юбка',
    'Платье',
    'Рубашка',
    'Жакет',
    'Блуза',
    'Пальто зимнее',
    'Костюм'
);

CREATE TYPE cut_style_type AS ENUM (
    'Приталенный',
    'Прямой',
    'Свободный',
    'Классический',
    'Расклешённый'
);

CREATE TYPE applicability_type AS ENUM (
    'Основная',
    'Подкладочная',
    'Прокладочная',
    'Отделочная'
);

-- 2. Таблицы

CREATE TABLE atelier (
    atelier_id SERIAL       PRIMARY KEY,
    address    VARCHAR(200) NOT NULL,
```

```
    phone      VARCHAR(15)  NOT NULL
);

CREATE TABLE client (
    client_id  SERIAL       PRIMARY KEY,
    surname    VARCHAR(40)  NOT NULL,
    name       VARCHAR(15)  NOT NULL,
    patronymic VARCHAR(20),
    phone      VARCHAR(15)  NOT NULL,
    email      VARCHAR(320),
    birth_date DATE         NOT NULL
);

CREATE TABLE atelier_client (
    client_id  INT NOT NULL REFERENCES client(client_id)   ON DELETE CASCADE,
    atelier_id INT NOT NULL REFERENCES atelier(atelier_id) ON DELETE CASCADE,
    PRIMARY KEY (client_id, atelier_id)
);

CREATE TABLE administrator (
    admin_id   SERIAL       PRIMARY KEY,
    surname    VARCHAR(40)  NOT NULL,
    name       VARCHAR(15)  NOT NULL,
    patronymic VARCHAR(20),
    phone      VARCHAR(15)  NOT NULL,
    email      VARCHAR(320) NOT NULL,
    birth_date DATE         NOT NULL,
    atelier_id INT          NOT NULL REFERENCES atelier(atelier_id)
);

CREATE TABLE tailor (
    tailor_id  SERIAL       PRIMARY KEY,
    surname    VARCHAR(40)  NOT NULL,
    name       VARCHAR(15)  NOT NULL,
    patronymic VARCHAR(20),
    phone      VARCHAR(15)  NOT NULL,
    email      VARCHAR(320),
    birth_date DATE         NOT NULL,
    atelier_id INT          NOT NULL REFERENCES atelier(atelier_id)
);

CREATE TABLE "order" (
    order_id        SERIAL        PRIMARY KEY,
    order_date      DATE          NOT NULL,
    completion_date DATE,
    status          order_status  NOT NULL DEFAULT 'Принят',
    total_cost      DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    client_id       INT           NOT NULL REFERENCES client(client_id),
```

```
93      tailor_id       INT             NOT NULL REFERENCES tailor(tailor_id),
94      admin_id        INT             NOT NULL REFERENCES administrator(admin_id)
95  );
96
97  CREATE TABLE garment (
98      garment_id   SERIAL         PRIMARY KEY,
99      product_type garment_type   NOT NULL,
100     cut_style    cut_style_type NOT NULL,
101     order_id     INT            NOT NULL REFERENCES "order"(order_id),
102     tailor_id    INT            NOT NULL REFERENCES tailor(tailor_id)
103 );
104
105 CREATE TABLE fabric (
106     fabric_id     SERIAL             PRIMARY KEY,
107     article       VARCHAR(20)        NOT NULL,
108     manufacturer  VARCHAR(60)        NOT NULL,
109     composition   VARCHAR(75)        NOT NULL,
110     meters_used   DECIMAL(10,2)      NOT NULL,
111     color         VARCHAR(25)        NOT NULL,
112     applicability applicability_type NOT NULL DEFAULT 'Основная'
113 );
114
115 CREATE TABLE fabric_garment (
116     garment_id INT NOT NULL REFERENCES garment(garment_id) ON DELETE CASCADE,
117     fabric_id  INT NOT NULL REFERENCES fabric(fabric_id)   ON DELETE CASCADE,
118     PRIMARY KEY (garment_id, fabric_id)
119 );
120
121 CREATE TABLE accessories (
122     accessory_id  SERIAL      PRIMARY KEY,
123     type          VARCHAR(30) NOT NULL,
124     material      VARCHAR(40) NOT NULL,
125     manufacturer  VARCHAR(60) NOT NULL,
126     article       VARCHAR(20) NOT NULL,
127     quantity_used INT         NOT NULL
128 );
129
130 CREATE TABLE accessories_garment (
131     garment_id   INT NOT NULL REFERENCES garment(garment_id)      ON DELETE
        ↪  CASCADE,
132     accessory_id INT NOT NULL REFERENCES accessories(accessory_id) ON DELETE
        ↪  CASCADE,
133     PRIMARY KEY (garment_id, accessory_id)
134 );
```

Листинг 4. Программа генерации базы данных

7 Приложение Б: Программа генерации записей базы данных

```cpp
#include <libpq-fe.h>

#include <algorithm>
#include <cstdlib>
#include <ctime>
#include <iostream>
#include <random>
#include <sstream>
#include <stdexcept>
#include <string>
#include <unordered_set>
#include <vector>


static const char* CONN_STR =
    "host=localhost port=5432 dbname=atelier user=postgres password=postgres";

struct DB {
    PGconn* conn = nullptr;

    explicit DB(const char* cs) {
        conn = PQconnectdb(cs);
        if (PQstatus(conn) != CONNECTION_OK)
            throw std::runtime_error(std::string("Ошибка подключения: ") +
                                     PQerrorMessage(conn));
        std::cout << "[OK] Подключено к БД\n";
    }
    ~DB() { if (conn) PQfinish(conn); }

    // Выполнить запрос без возвращаемых строк
    void exec(const std::string& sql) {
        PGresult* r = PQexec(conn, sql.c_str());
        if (PQresultStatus(r) != PGRES_COMMAND_OK) {
            std::string err = PQerrorMessage(conn);
            PQclear(r);
            throw std::runtime_error("exec() error: " + err + "\nSQL: " + sql);
        }
        PQclear(r);
    }

    // Выполнить запрос и вернуть первое поле первой строки
    std::string scalar(const std::string& sql) {
        PGresult* r = PQexec(conn, sql.c_str());
        std::string val;
```

```
45          if (PQresultStatus(r) == PGRES_TUPLES_OK && PQntuples(r) > 0)
46              val = PQgetvalue(r, 0, 0);
47          PQclear(r);
48          return val;
49      }
50
51      void begin()    { exec("BEGIN"); }
52      void commit()   { exec("COMMIT"); }
53      void rollback() { exec("ROLLBACK"); }
54  };
55
56  static std::mt19937 rng(static_cast<unsigned>(std::time(nullptr)));
57
58  int randInt(int lo, int hi) {                          // [lo, hi]
59      return std::uniform_int_distribution<int>(lo, hi)(rng);
60  }
61  double randDouble(double lo, double hi) {
62      return std::uniform_real_distribution<double>(lo, hi)(rng);
63  }
64  template<typename T>
65  const T& pick(const std::vector<T>& v) {
66      return v[randInt(0, static_cast<int>(v.size()) - 1)];
67  }
68
69  //  Вспомогательные генераторы строк
70  std::string randomPhone() {
71      std::ostringstream oss;
72      oss << "+7" << randInt(900, 999)
73          << std::to_string(1000000 + randInt(0, 8999999));
74      return oss.str();
75  }
76
77  std::string randomDate(int y1, int y2) {
78      // Простая генерация: год + день года
79      int y = randInt(y1, y2);
80      int m = randInt(1, 12);
81      int maxD = 28;
82      if (m == 2) maxD = 28;
83      else if (m == 4 || m == 6 || m == 9 || m == 11) maxD = 30;
84      else maxD = 31;
85      int d = randInt(1, maxD);
86      char buf[12];
87      snprintf(buf, sizeof(buf), "%04d-%02d-%02d", y, m, d);
88      return buf;
89  }
90
91  // Экранирование одиночных кавычек для SQL-строк
92  std::string esc(const std::string& s) {
```

```
    std::string out;
    out.reserve(s.size());
    for (char c : s) {
        if (c == '\'') out += "''";
        else out += c;
    }
    return out;
}

std::string q(const std::string& s)  { return "'" + esc(s) + "'"; }
std::string qn(const std::string& s) { return s.empty() ? "NULL" : q(s); }

// Фамилии
const std::vector<std::string> SURNAMES_M = {
    "Иванов","Петров","Сидоров","Козлов","Новиков","Морозов","Волков",
    "Лебедев","Семёнов","Зайцев","Попов","Соколов","Михайлов","Фёдоров",
    "Орлов","Громов","Беляев","Тихонов","Куликов","Романов","Макаров",
    "Виноградов","Ковалёв","Герасимов","Никитин","Борисов","Зубов","Карпов"
};
const std::vector<std::string> SURNAMES_F = {
    "Иванова","Петрова","Сидорова","Козлова","Новикова","Морозова","Волкова",
    "Лебедева","Семёнова","Зайцева","Попова","Соколова","Михайлова","Фёдорова",
    "Орлова","Громова","Беляева","Тихонова","Куликова","Романова","Макарова",
    "Виноградова","Ковалёва","Герасимова","Никитина","Борисова","Зубова","Карпова"
};
const std::vector<std::string> NAMES_M = {
    "Алексей","Дмитрий","Сергей","Андрей","Николай","Владимир","Михаил",
    "Иван","Павел","Артём","Константин","Геннадий","Руслан","Роман","Игорь"
};
const std::vector<std::string> NAMES_F = {
    "Анна","Мария","Елена","Ольга","Наталья","Татьяна","Ирина","Светлана",
    "Юлия","Екатерина","Валентина","Людмила","Вероника","Ксения","Алина"
};
const std::vector<std::string> PATRONYMICS = {
    "Александрович","Сергеевич","Иванович","Николаевич","Дмитриевич",
    "Андреевич","Викторович","Максимович","Геннадьевич","Константинович",
    "Александровна","Сергеевна","Ивановна","Николаевна","Дмитриевна",
    "Андреевна","Викторовна","Максимовна","Геннадьевна","Константиновна"
};

const std::vector<std::string> ORDER_STATUSES = {
    "Принят","В работе","Готов","Выдан","Отменён"
};
const std::vector<std::string> GARMENT_TYPES = {
    "Пальто","Пиджак","Брюки","Юбка","Платье",
    "Рубашка","Жакет","Блуза","Пальто зимнее","Костюм"
};
const std::vector<std::string> CUT_STYLES = {
```

```
    "Приталенный","Прямой","Свободный","Классический","Расклешённый"
};
const std::vector<std::string> APPLICABILITIES = {
    "Основная","Подкладочная","Прокладочная","Отделочная"
};

const std::vector<std::string> FABRIC_MANUFACTURERS = {
    "Текстиль-Про","Иваново Текстиль","FabricWorld","Euro Textile",
    "СибТкань","КамаТекс","НордФэбрик","ЮгТекстиль","МосТкань","ПолиТекс"
};
const std::vector<std::string> FABRIC_COLORS = {
    "Белый","Чёрный","Серый","Бежевый","Синий","Тёмно-синий","Красный",
    "Бордовый","Зелёный","Хаки","Коричневый","Кремовый","Молочный",
    "Антрацит","Графит","Терракот","Горчичный","Пудровый","Лавандовый","Изумрудный"
};
const std::vector<std::string> FABRIC_COMPOSITIONS = {
    "100% шерсть","100% хлопок","100% лён",
    "65% полиэстер, 35% вискоза","50% шерсть, 50% полиэстер",
    "70% хлопок, 30% полиэстер","100% вискоза","80% акрил, 20% шерсть",
    "60% лён, 40% хлопок","95% полиэстер, 5% эластан"
};

const std::vector<std::string> ACC_TYPES = {
    "Пуговица","Молния","Крючок","Кнопка","Липучка",
    "Заклёпка","Шлёвка","Петля","Бретель","Застёжка"
};
const std::vector<std::string> ACC_MATERIALS = {
    "Металл","Пластик","Нейлон","Хлопок","Кожа",
    "Латунь","Нержавеющая сталь","Акрил","Перламутр","Дерево"
};
const std::vector<std::string> ACC_MANUFACTURERS = {
    "Prym","YKK","Coats","КнопкаМастер","ФурнитураПлюс",
    "ButtonCraft","ZipWorld","MetalArt","SewPro","AccessoryHub"
};

struct AtelierData { std::string address; std::string phone; };
const std::vector<AtelierData> ATELIERS = {
    {"Москва, ул. Тверская, д. 12, оф. 3",          "+74951234567"},
    {"Москва, ул. Арбат, д. 25, оф. 8",             "+74959876543"},
    {"Санкт-Петербург, Невский пр., д. 47",          "+78122345678"},
    {"Санкт-Петербург, ул. Рубинштейна, д. 9",       "+78129876543"},
    {"Новосибирск, Красный пр., д. 30, оф. 12",      "+73832345678"},
    {"Екатеринбург, ул. Ленина, д. 8, оф. 2",        "+73432345678"},
    {"Казань, ул. Баумана, д. 51",                   "+78432345678"},
    {"Нижний Новгород, ул. Большая Покровская, д. 17","+78312345678"},
    {"Краснодар, ул. Красная, д. 44, оф. 5",         "+78612345678"},
    {"Ростов-на-Дону, пр. Ворошиловский, д. 22",     "+78632345678"},
```

```
};


struct Person {
    std::string surname, name, patronymic;
};

Person randomPerson(bool allowNullPatronymic = true) {
    bool female = randInt(0, 1);
    Person p;
    p.surname    = female ? pick(SURNAMES_F)   : pick(SURNAMES_M);
    p.name       = female ? pick(NAMES_F)      : pick(NAMES_M);
    p.patronymic = (allowNullPatronymic && randInt(0, 9) == 0) ? "" :
     ↪  pick(PATRONYMICS);
    return p;
}

std::string randomEmail(const Person& p, int idx, bool nullable = true) {
    if (nullable && randInt(0, 2) == 0) return "";
    std::ostringstream oss;
    oss << "user" << idx << "@example.ru";
    return oss.str();
}

void fillAteliers(DB& db) {
    db.begin();
    for (auto& a : ATELIERS) {
        db.exec("INSERT INTO atelier (address, phone) VALUES ("
                + q(a.address) + "," + q(a.phone) + ")");
    }
    db.commit();
}

void fillTailors(DB& db, int atelierCount) {
    int total = 0;
    db.begin();
    for (int a = 1; a <= atelierCount; ++a) {
        int cnt = randInt(10, 15);
        for (int i = 0; i < cnt; ++i) {
            Person p = randomPerson();
            std::string email = randomEmail(p, total + i, true);
            db.exec("INSERT INTO tailor "
                    "(surname,name,patronymic,phone,email,birth_date,atelier_id)
                     ↪  VALUES ("
                    + q(p.surname) + "," + q(p.name) + "," + qn(p.patronymic) +
                     ↪  ","
                    + q(randomPhone()) + "," + qn(email) + ","
                    + q(randomDate(1970, 2000)) + "," + std::to_string(a) + ")");
```

```
233         }
234         total += cnt;
235     }
236     db.commit();
237 }
238
239 void fillAdmins(DB& db, int atelierCount) {
240     int total = 0;
241     db.begin();
242     for (int a = 1; a <= atelierCount; ++a) {
243         int cnt = randInt(5, 7);
244         for (int i = 0; i < cnt; ++i) {
245             Person p = randomPerson(false);
246             std::string email = "admin" + std::to_string(total + i) +
                ↪  "@atelier.ru";
247             db.exec("INSERT INTO administrator "
248                     "(surname,name,patronymic,phone,email,birth_date,atelier_id)
                        ↪  VALUES ("
249                     + q(p.surname) + "," + q(p.name) + "," + qn(p.patronymic) +
                        ↪  ","
250                     + q(randomPhone()) + "," + q(email) + ","
251                     + q(randomDate(1975, 1995)) + "," + std::to_string(a) + ")");
252         }
253         total += cnt;
254     }
255     db.commit();
256 }
257
258 void fillClients(DB& db, int count) {
259     db.begin();
260     for (int i = 1; i <= count; ++i) {
261         Person p = randomPerson();
262         std::string email = randomEmail(p, i, true);
263         db.exec("INSERT INTO client "
264                 "(surname,name,patronymic,phone,email,birth_date) VALUES ("
265                 + q(p.surname) + "," + q(p.name) + "," + qn(p.patronymic) + ","
266                 + q(randomPhone()) + "," + qn(email) + ","
267                 + q(randomDate(1950, 2005)) + ")");
268         if (i % 100 == 0) { db.commit(); db.begin(); }
269     }
270     db.commit();
271 }
272
273 void fillAtelierClient(DB& db, int clientCount, int atelierCount) {
274     db.begin();
275     for (int c = 1; c <= clientCount; ++c) {
276         int a1 = randInt(1, atelierCount);
277         db.exec("INSERT INTO atelier_client VALUES ("
```

```
278                 + std::to_string(c) + "," + std::to_string(a1) + ") ON CONFLICT DO
                     ↪  NOTHING");
279         if (randInt(0, 9) < 3) {
280             int a2 = randInt(1, atelierCount);
281             if (a2 != a1) {
282                 db.exec("INSERT INTO atelier_client VALUES ("
283                         + std::to_string(c) + "," + std::to_string(a2) + ") ON
                          ↪  CONFLICT DO NOTHING");
284             }
285         }
286     }
287     db.commit();
288 }
289
290 void fillOrders(DB& db, int clientCount) {
291     PGresult* r = PQexec(db.conn, "SELECT tailor_id, atelier_id FROM tailor");
292     int tailorRows = PQntuples(r);
293     struct TailorInfo { int id; int atelierID; };
294     std::vector<TailorInfo> tailors(tailorRows);
295     for (int i = 0; i < tailorRows; ++i) {
296         tailors[i].id        = std::stoi(PQgetvalue(r, i, 0));
297         tailors[i].atelierID = std::stoi(PQgetvalue(r, i, 1));
298     }
299     PQclear(r);
300
301     r = PQexec(db.conn, "SELECT admin_id, atelier_id FROM administrator");
302     std::vector<std::pair<int,int>> admins;
303     for (int i = 0; i < PQntuples(r); ++i)
304         admins.push_back({std::stoi(PQgetvalue(r,i,0)),
             ↪  std::stoi(PQgetvalue(r,i,1))});
305     PQclear(r);
306
307     db.begin();
308     for (int c = 1; c <= clientCount; ++c) {
309         int cnt = randInt(1, 10);
310         for (int i = 0; i < cnt; ++i) {
311             auto& t = pick(tailors);
312             int adminID = -1;
313             std::vector<int> sameAtelier;
314             for (auto& a : admins)
315                 if (a.second == t.atelierID) sameAtelier.push_back(a.first);
316             if (!sameAtelier.empty())
317                 adminID = sameAtelier[randInt(0, (int)sameAtelier.size()-1)];
318             else
319                 adminID = admins[randInt(0,(int)admins.size()-1)].first;
320
321             std::string status = pick(ORDER_STATUSES);
322             std::string oDate  = randomDate(2021, 2024);
```

```
323             std::string cDate  = "NULL";
324             if (status == "Готов" || status == "Выдан")
325                 cDate = q(randomDate(2022, 2025));
326             double cost = 500.0 + randDouble(0, 49500);
327             std::ostringstream sql;
328             sql << "INSERT INTO \"order\" "
329 
                    ↪  "(order_date,completion_date,status,total_cost,client_id,tailor_id
                    ↪  VALUES ("
330                 << q(oDate) << "," << cDate << "," << q(status) << ","
331                 << std::fixed << cost << "," << c << "," << t.id << "," << adminID
                     ↪  << ")";
332             db.exec(sql.str());
333         }
334         if (c % 100 == 0) { db.commit(); db.begin(); }
335     }
336     db.commit();
337 }
338 
339 void fillGarments(DB& db) {
340     PGresult* r = PQexec(db.conn, "SELECT order_id, tailor_id FROM \"order\"");
341     int rows = PQntuples(r);
342     struct OrderInfo { int id; int tailorID; };
343     std::vector<OrderInfo> orders(rows);
344     for (int i = 0; i < rows; ++i) {
345         orders[i].id       = std::stoi(PQgetvalue(r,i,0));
346         orders[i].tailorID = std::stoi(PQgetvalue(r,i,1));
347     }
348     PQclear(r);
349 
350     int total = 0;
351     db.begin();
352     for (auto& o : orders) {
353         int cnt = randInt(1, 5);
354         for (int i = 0; i < cnt; ++i) {
355             db.exec("INSERT INTO garment
                     ↪  (product_type,cut_style,order_id,tailor_id) VALUES ("
356                     + q(pick(GARMENT_TYPES)) + "," + q(pick(CUT_STYLES)) + ","
357                     + std::to_string(o.id) + "," + std::to_string(o.tailorID) +
                         ↪  ")");
358             ++total;
359         }
360         if (total % 500 == 0) { db.commit(); db.begin(); }
361     }
362     db.commit();
363 }
364 
365 void fillFabrics(DB& db, int count) {
```

```
366     db.begin();
367     for (int i = 1; i <= count; ++i) {
368         char article[24];
369         snprintf(article, sizeof(article), "TKN-%05d", i);
370         std::ostringstream meters;
371         meters << std::fixed; meters.precision(2);
372         meters << (0.5 + randDouble(0, 9.5));
373         db.exec("INSERT INTO fabric "
374 
                 ↪  "(article,manufacturer,composition,meters_used,color,applicability)
                 ↪  VALUES ("
375                 + q(article) + "," + q(pick(FABRIC_MANUFACTURERS)) + ","
376                 + q(pick(FABRIC_COMPOSITIONS)) + "," + meters.str() + ","
377                 + q(pick(FABRIC_COLORS)) + "," + q(pick(APPLICABILITIES)) + ")");
378     }
379     db.commit();
380 }
381 
382 void fillAccessories(DB& db, int count) {
383     db.begin();
384     for (int i = 1; i <= count; ++i) {
385         char article[24];
386         snprintf(article, sizeof(article), "ACC-%05d", i);
387         db.exec("INSERT INTO accessories "
388                 "(type,material,manufacturer,article,quantity_used) VALUES ("
389                 + q(pick(ACC_TYPES)) + "," + q(pick(ACC_MATERIALS)) + ","
390                 + q(pick(ACC_MANUFACTURERS)) + "," + q(article) + ","
391                 + std::to_string(randInt(1, 50)) + ")");
392     }
393     db.commit();
394 }
395 
396 void fillLinks(DB& db, int fabricCount, int accessoryCount) {
397     PGresult* r = PQexec(db.conn, "SELECT garment_id FROM garment ORDER BY
        ↪  garment_id");
398     int rows = PQntuples(r);
399     std::vector<int> garmentIDs(rows);
400     for (int i = 0; i < rows; ++i)
401         garmentIDs[i] = std::stoi(PQgetvalue(r, i, 0));
402     PQclear(r);
403 
404     db.begin();
405     int batch = 0;
406     for (int gid : garmentIDs) {
407         {
408             int cnt = randInt(3, 5);
409             std::unordered_set<int> used;
410             while ((int)used.size() < cnt) {
```

```
411                 int fid = randInt(1, fabricCount);
412                 if (used.insert(fid).second) {
413                     db.exec("INSERT INTO fabric_garment VALUES ("
414                             + std::to_string(gid) + "," + std::to_string(fid) + ")
                             ↪  ON CONFLICT DO NOTHING");
415                 }
416             }
417         }
418         {
419             int cnt = randInt(10, 25);
420             std::unordered_set<int> used;
421             while ((int)used.size() < cnt) {
422                 int aid = randInt(1, accessoryCount);
423                 if (used.insert(aid).second) {
424                     db.exec("INSERT INTO accessories_garment VALUES ("
425                             + std::to_string(gid) + "," + std::to_string(aid) + ")
                             ↪  ON CONFLICT DO NOTHING");
426                 }
427             }
428         }
429         if (++batch % 200 == 0) { db.commit(); db.begin(); }
430     }
431     db.commit();
432 }
433
434 int main() {
435     try {
436         DB db(CONN_STR);
437         fillAteliers(db);
438         fillTailors(db,     (int)ATELIERS.size());
439         fillAdmins(db,      (int)ATELIERS.size());
440         fillClients(db,     1000);
441         fillAtelierClient(db, 1000, (int)ATELIERS.size());
442         fillOrders(db,      1000);
443         fillGarments(db);
444         fillFabrics(db,     500);
445         fillAccessories(db, 500);
446         fillLinks(db,       500, 500);
447         std::cout << "\n[ГОТОВО] База данных успешно заполнена.\n";
448     }
449     catch (const std::exception& ex) {
450         std::cerr << "\n[ОШИБКА] " << ex.what() << "\n";
451         return 1;
452     }
453     return 0;
454 }
```

Листинг 5. Программа генерации записей базы данных на C++